

PROYECTO FINAL DE CICLO DAW A DISTANCIA (2025-2026)

I.E.S. VIRGEN DE LA PAZ



ASOCIACIÓN DEPORTIVA DE LA PAZ



PONENTES: María Parreño Castillejo, Francisco José
Tieso Muñoz y Raúl García Gómez

Índice de memoria de proyecto

1. Justificación y objetivos.	3
2. Introducción.	3
2.1 Metodología SCRUM.	4
2.2 Tecnologías Utilizadas	5
2.2.1-FIREBASE	5
2.2.2-GITHUB	6
2.2.3-NODE.JS	7
2.2.4-TailwindCSS	8
2.2.5-Express JS	10
2.2.6- JWT	11
2.2.7- FRAMEWORKS	11
2.2.7.1-VisualStudio Code	12
2.2.7.2-KIRO	12
2.2.8-Inteligencia Artificial	13
2.2.8.1-GEMINI	13
2.2.8.2-CHATGPT	14
2.2.8.3-Copilot	14
3. Metodología y desarrollo del proyecto.	15
3.1 Historias de usuario	16
3.2 Plan de Proyecto.	17
3.3 Arquitectura de la aplicación. Esquema de los componentes con tecnologías asociadas	21
3.4 Diseño de la aplicación.	24
3.4.1 Diseño “Frontend”.	34
3.4.2 Diseño “Backend”.	37
3.4.2.1 Diseño BBDD.	38
3.5 Pruebas.	45
3.6 Despliegue de la aplicación.	45
4. Resultados y discusión.	48
5. Conclusiones.	49
6. Bibliografía y referencias.	49
Anexos.	52
A. Manual de identidad visual	52
B. Diagrama de clases	52
C. Documentación de la API	52

1. Justificación y objetivos.

El pasado mes de septiembre después de la creación del grupo de trabajo para el proyecto final de grado, compuesto por tres personas (María Parreño Castillejo, Francisco José Tieso Muñoz y Raúl García Gómez).

Decidimos crear un proyecto común llamado Portal Virtual Asociación Deportiva de la Paz.

Se suponía que habíamos recibido como desarrolladores un encargo de una asociación deportiva, dicha asociación estaba interesada en gestionar diferentes ligas y competiciones de futbol y baloncesto.

Desarrollamos un resumen de las especificaciones que queríamos incluir en el proyecto desde el principio.

Al poco tiempo, ni que decir tiene, desistimos del baloncesto y nos centramos solo en el futbol, dada la enorme complejidad que nos suponía inicialmente todo el desarrollo y con la perspectiva puesta en que en caso de funcionar bien la aplicación se podría escalar a cualquier deporte o actividad.

Decidimos que debía suponernos un reto, por eso inicialmente pensamos en utilizar diferentes tecnologías a las aprendidas en el Módulo a la vez que nos debía de permitir potenciar las habilidades ya adquiridas.

Como reto nos suponía entender el diseño desde cero de la aplicación con la clara utilización por parte de usuarios con diferentes funciones y roles que debían desempeñar diferentes actividades y relacionarse de forma diferente con la aplicación web.

La aplicación debía ser funcional, intuitiva, con un diseño “responsive” y atractiva visualmente.

2. Introducción

En este punto nos centraremos en exponer lo que queremos que realice la aplicación y después iniciaremos una explicación de cada apartado para que el lector comprenda los pasos que se han dado.

Como se ha explicado en el punto anterior la aplicación consiste en una Aplicación Web que se ocupa de la gestión de las ligas de futbol de diferentes categorías y tipos, además claro esta de publicar los resultados.

La web debe tener dos accesos, por un lado el de usuarios generales o anónimos que acceden a la web solo para ver los datos que allí se exponen (clasificaciones y resultados) y los usuarios logeados que desempeñan un rol (administradores, delegados, entrenadores y árbitros).

Por un lado se deben poder crear equipos, competiciones y jugadores según las necesidades. La web tiene un sistema de alta de usuarios asistido, es decir, para darse de alta es necesario contactar con los administradores del sitio y solicitarles el alta como usuario, no se puede hacer directamente por parte del usuario.

Según el rol de usuario al logearse se accede a una serie de paneles mediante archivos html que permiten la gestión CRUD de los datos según el rol.

Existe una versión web disponible sin ningún tipo de login que permite la visualización de archivos html de información general ni posibilidad de modificación de datos, solo visualización.

2.1 Metodología SCRUM

SCRUM es una metodología ágil utilizada para gestionar proyectos completos, se utiliza en desarrollo de software y su objetivo es entregar valor de forma rápida, continua y adaptable. Sus orígenes datan de los años 80 y 90 como respuesta a un problema muy común en los desarrollos, que tradicionalmente eran muy lentos, rígidos y fallaban con frecuencia. Inspirado en un artículo de Hirotaka Takeuchi publicado en 1986 en Harvard Business Review, en el que estudiaba el rendimiento de una jugada de rugby llamada scrum, en la que todos avanzan juntos como una única unidad.

Jeff Sutherland y Ken Schwaber crearon la metodología SCRUM, la presentaron en 1995 en la conferencia OOPSLA, en 2001 crearon el Manifiesto Ágil y en 2010 finalmente publicaron la primera versión de la Guía SCRUM. En cierta forma debió ser una gran evolución, SCRUM permite adaptarse rápidamente a los cambios, ajustando prioridades, cambiando los requisitos y corrigiendo errores de forma rápida.

Fomenta los equipos autónomos y responsables, permitiendo la entrega de valor desde el primer Sprint, aporta un punto de transparencia total y proporciona una mejora continua.

Y tras esta breve explicación en este proyecto decidimos implementar un desarrollo mediante SCRUM desde el principio y subir el proyecto a la plataforma JIRA a la que dimos acceso a nuestro tutor de proyecto.

Cuando tuvimos claro lo que íbamos a hacer y cómo pretendíamos hacerlo empezamos a trabajar.

El primer paso no incluido en JIRA sería el diseño del proyecto y las tareas más generales, incluyendo el proceso de alta y registro de todas las tareas.

Nuestros comentarios y observaciones sobre el SCRUM.

Al utilizar la metodología SCRUM nuestro objetivo en cada SPRINT era poder hacer entrega a un supuesto cliente de un producto mínimamente viable y funcional.

El mayor cambio efectuado en la planificación fue el realizado en los roles de usuario y el sistema de autenticación, gracias a la observación de nuestro tutor Jorge Iván Rodríguez nos mostraron algo que no habíamos visto ningún integrante del equipo de proyecto. Nuestro sistema de autenticación era muy básico, la permanencia de la sesión de usuario solo se realizaba a través de la creación de una cookie en el navegador, que contenía los datos de usuario. Al encontrarnos con este reto decidimos abordarlo de la forma más segura posible y decidimos implementar JWT (JSON WEB TOKEN) basado en tokens firmados y no en sesiones de servidor, gracias a este punto conseguimos dar una seguridad mínima al acceso a la web y a los diferentes roles, impidiendo el acceso no autorizado.

El segundo punto importante no incluido en los SPRINT porque surgió sobre la marcha y nos retrasó considerablemente dada la cantidad de horas de trabajo que tuvimos que dedicarle fue la creación de nuestra propia API para manejar la base de datos, otra muy buena aportación de nuestro tutor de proyecto.

Firebase provee de una API básica para la interconexión con la base de datos en la nube, que habíamos utilizado hasta el Sprint 2, gracias a las observaciones del tutor a partir del Sprint 3 nos

pusimos a trabajar en diseñar nuestra propia API, es decir, los controladores, rutas, servicios, utilidades, etc. A priori este paso podría parecer difícil pero lo más difícil fue retirar todas las llamadas a la primera API de la web y asegurarnos de que era nuestra API la que controlaba el proyecto, fue una tarea ardua. Aun utilizando herramientas de IA (Inteligencia Artificial) invertimos muchas horas en poder librarnos de esta primera opción (la API de Firebase) que no nos permitía tener un buen control sobre el proyecto.

La creación de la base de datos y su acceso a través de los correspondientes archivos JSON a priori se presento como una tarea difícil pero en la practica fue bastante sencillo porque se utilizo la interfaz gráfica de FirebaseDatabase que es muy intuitiva (creada y mantenida por Google).

La metodología SCRUM nos resulto muy útil pero el mantenimiento de la información actualizada en la plataforma JIRA ha sido muy costoso. Lo bueno, te permite controlar todo el proyecto desde el primer minuto, al ser solo tres miembros en el equipo el papel de ProductOwner prácticamente lo ha desempeñado nuestro tutor (gracias a su colaboración) porque no teníamos más recursos.

2.2 Tecnologías Utilizadas

En este apartado vamos a incluir todas las tecnologías incluidas en el proyecto.



URL: <https://firebase.google.com>

2.2.1-FIREBASE (HOSTING DEL PROYECTO Y LA BASE DE DATOS)

Para empezar el proyecto se encuentra alojado en la plataforma FIREBASE, se trata de una plataforma de desarrollo creada por Google que ofrece varias herramientas para la construcción de Aplicaciones web y móviles, sin tener que gestionar los servidores esta al 100% en Cloud, permite conexión con GitHub y actualiza el repositorio de Firebase de forma automática.

A través de su plataforma FIREBASESTUDIO empezamos a crear la base o estructura del proyecto y en adelante nos dimos cuenta que FIREBASESTUDIO no es más que un panel de administración WEB, dadas sus grandes limitaciones y graves riesgos, decidimos dejar de utilizarlo y centrarnos en el uso de Frameworks locales como KIRO y VISUALSTUDIO CODE para la creación del proyecto y todos sus archivos.

Se realizo un breve desligue inicial con una maqueta o boceto del sitio web y probamos que era funcional y estable, a partir de ahí, nos centramos en el desarrollo en local hasta la finalización del mismo cuando nos dispusimos para su despliegue final.

Firebase nos proporciona tres cosas, dos son totalmente Free y la tercera es de pago por uso.

- Servidor WEB (similar a cualquier Servidor Apache).
- Servidor de BBDD (NOSQL, se trata de un servidor de Base de Datos no relacional)
- Servidor Node.js (es de pago, pero fundamental para el acceso a la Base de Datos)

Nos ha permitido el despliegue inmediato de la aplicación de forma pública mediante la siguiente URL: <https://de-la-paz-42126976-5cd58.web.app>

Este servicio de HOSTING es muy recomendable para el desarrollo de proyectos, incorpora GEMINI al desarrollo de forma nativa. El único inconveniente es el servidor node.js. Para poder tenerlo hay que activar el plan de crédito en Google que solo contiene 300 \$, es limitado. También dispone de analítica web (la de Google) entorno para test A/B, autenticación de Google (mucho más robusta pero poco práctica para el proyecto), diferentes tipos y modelos de servidores entre ellos plataformas para Android y Mac. Como herramienta de despliegue del proyecto nos pareció muy buena, pero el despliegue debería esperar hasta el final.



URL: <https://github.com/>

2.2.2-GITHUB

Es una plataforma en la nube donde se pueden trabajar de forma colaborativa varios desarrolladores compartiendo y colaborando en el código. Git como tal es solo el sistema de control de versiones. A este repositorio también dimos acceso a nuestro tutor para poder supervisar el proyecto. Los archivos de la web están subidos al repositorio: <https://github.com/FranTieso/De-la-Paz.git> NO ES UN REPOSITORIO PÚBLICO, adjuntaremos el material en el USB que entregamos. Esta plataforma nos ha permitido el control de versiones y la posibilidad de trabajar de forma sincronizada.

Por ejemplo, como ya hemos explicado anteriormente inicialmente se utilizó la API de Firebase, hasta que la nuestra no estuvo 100% disponible no empezamos a hacer la migración de una a otra. A pesar de lo complejo de la tarea la utilización de diferentes Inteligencias Artificiales nos ha permitido hacerlo de una forma rápida (tardamos un día aproximadamente).

Por otro lado también sabíamos lo que estábamos haciendo cada uno de los integrantes del equipo, solo había que revisar el historial de versiones.

Intentamos la creación de diferentes Branch o ramas, pero resultó en un desastre absoluto, por consiguiente aunque somos conscientes de que no es lo más ortodoxo decidimos trabajar todos sobre el Branch MAIN.

Todo esto nos ha permitido a pesar de no estar en contacto diariamente por diferentes motivos conocer cuál era el trabajo realizado por cada uno y el trabajo pendiente por hacer.



URL: <https://nodejs.org/es>

2.2.3-NODE.JS

Es un entorno de ejecución que permite ejecutar JavaScript fuera del navegador, habitualmente en un servidor NODE.JS.

Nos permite convertir JavaScript en un lenguaje válido para ser ejecutado en el lado del servidor (BackEnd), lo cual nos ha permitido crear APIs, servidores Web, automatizar tareas y procesos o crear y ejecutar micro servicios entre otros. Es asíncrono y orientado a Eventos.

De hecho toda la web se podría haber creado con esta tecnología, lo cual nos pareció demasiado aventurero y arriesgado, por eso la limitamos solo al enlace entre la base de datos y la web.

Nos ha permitido acercar la base de datos a la web, ejecutando nuestra API creada 100% en JavaScript enlazando las funciones CRUD (Create, Read, Update y Delete) y es el nexo entre la base de datos (servidor de base de datos no relacional) y la web (navegador del usuario).

Esta tecnología decidimos utilizarla desde el principio, no la hemos estudiado ni trabajado en el Módulo y nos pareció todo un reto que deseábamos abordar.

En el primer momento gracias a la utilización de otros Frameworks como Express y sobre todo a la utilización de Inteligencia Artificial (para el aprendizaje y después manipulación de archivos) hemos conseguido familiarizarnos con esta tecnología.

Más adelante explicaremos como interactúa la web con el servidor NODE.JS, sirva como adelanto estas líneas.

Básicamente el usuario a través del navegador hace una llamada a una API desde JavaScript a través de funciones incorporadas en el archivo HTML bien de forma directa o bien modularizada, estas funciones generan en el navegador del cliente una petición HTTP (GET, POST, DELETE o SET), una URL o endpoint, los headers o cabeceras, datos si fuera necesario (muy útiles en el caso de búsquedas) y cookies o tokens como JWT (fundamental en nuestro proyecto para darle un poco de seguridad) y procede a su envío al servidor.

El Servidor recibe la petición, bien con NODE.JS, DJANGO, LARAVEL, JAVA SPRING, etc la lee y procesa, validando los datos, comprobando permisos (muy importante a través de JWT) y ejecuta la lógica de la petición, en este caso acceder a la base de datos y procesar la petición (según el rol de usuario y su token tiene unos permisos de acceso, consulta, modificación o supresión a las diferentes colecciones de datos de la base de datos NOSQL y a sus campos).

El servidor de la base de datos procesa la petición y envía una respuesta en formato JSON o XML, en nuestro caso siempre será JSON porque nuestra API REST así lo necesita para poder trabajar con la respuesta del servidor de la base de datos, devuelve el JSON al servidor NODE.JS, gracias a la

ejecución de JavaScript en el servidor, se configura la web que va a visualizar el cliente y se prepara para su envío.

Por último, el servidor envía la respuesta HTML o JSON al navegador del cliente para que la procese dicho navegador, mientras pueda ver y seguir utilizando la aplicación web. En este caso cabe decir que el navegador actualiza la interfaz, al ser una conexión asíncrona puede o no actualizar toda la página web, eso dependerá de la solicitud enviada al servidor y de la respuesta de este.



URL: <https://tailwindcss.com/brand>

2.2.4-TAILWINDCSS

TailwindCSS es un framework de CSS basado en utilidades. En lugar de escribir nuestros propios archivos CSS importamos los de TailwindCSS, nos permite diseñar rápido creando clases listas para usar sin escribir el CSS manualmente con el consiguiente ahorro de tiempo.

Es similar a Bootstrap en PHP, Bootstrap sí lo hemos estudiado en el Módulo.

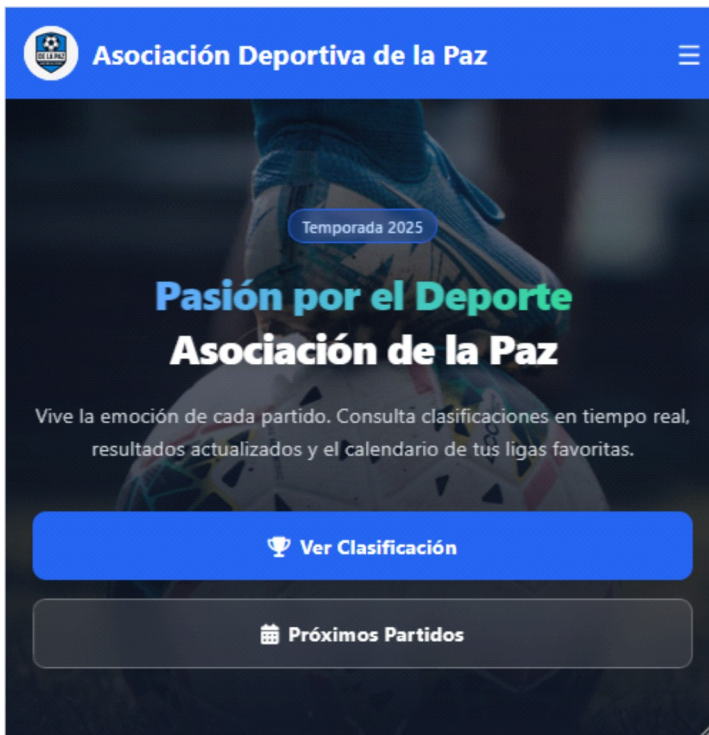
Mientras Bootstrap es más predefinido, tiene unos estilos propios y nos pareció menos flexible que TailwindCSS (esto es solo una opinión personal nuestra). TailwindCSS tiene una ventaja muy grande es 100% RESPONSIVE, permite la adaptación automática al tamaño de pantalla utilizado, es decir, que la web no se visualiza igual en un teléfono móvil, tablet o PC.

Sirva este ejemplo:

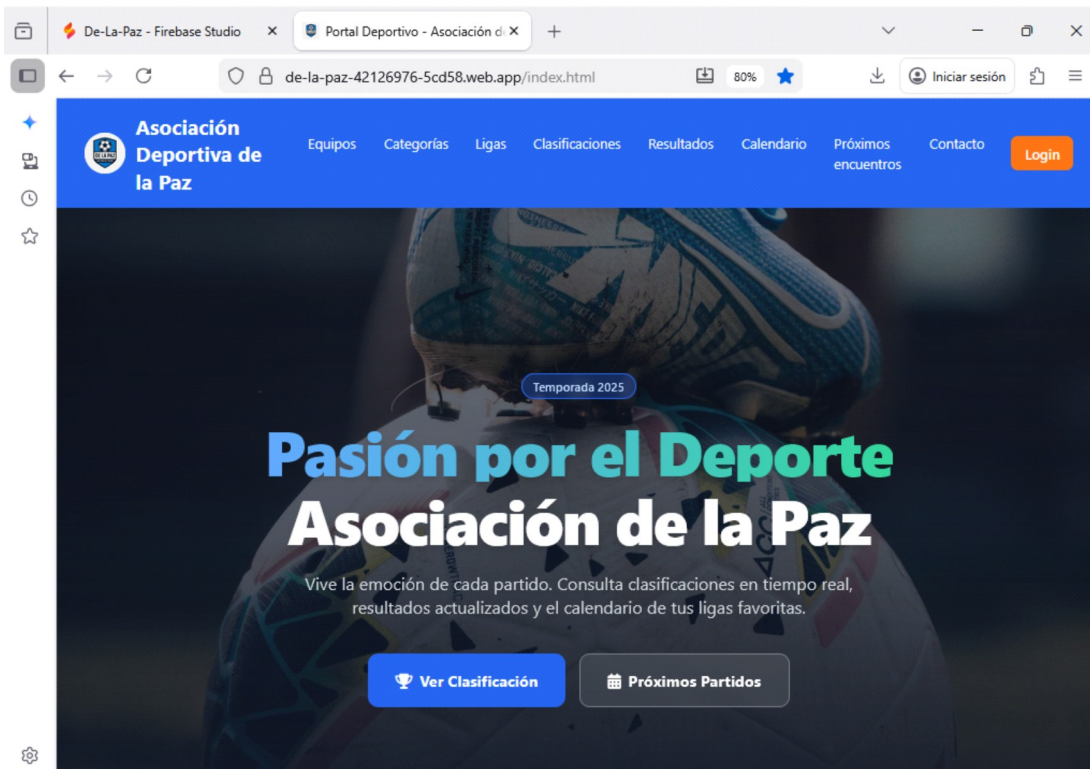
Versión Móvil:



Versión Tablet:



Versión PC:





URL: <https://expressjs.com/es/api.html>

2.2.5-Express JS

Es un framework para NODE.JS, permite la creación de APIs REST para servidores web y manejar rutas, peticiones y respuestas de forma muy sencilla.

Cuando decidimos crear la API de nuestra web a recomendación de nuestro tutor decidimos usar este framework (que también fue recomendación del tutor). Es una herramienta estándar para construir APIs en Node.JS de forma rápida y flexible. El trabajo más duro en esta parte fue adaptar todas las peticiones que ya existían en la web utilizando la API básica de Firebase y después de probar mucho, trasladarla a la web, donde nuevamente se tuvo que volver a probar.

Después de varios intentos y de revisar la API con herramientas de inteligencia artificial (Gemini, ChatGPT y Copilot), conseguimos que fuera estable y que nos permitiera el trabajo con la base de datos.

Conseguimos configurar nuestro propio servidor y compartirlo a través del GitHub (cada miembro del equipo tuvo que instalar un paquete adicional de software para poder crear el servidor en local que se configuraba gracias a los archivos del repositorio).

Activamos los middlewares, definimos rutas y por último arrancamos el servidor en local.

La API se encuentra en la carpeta del mismo nombre dentro del proyecto, dentro se encuentran las principales funciones de la web, a saber, la gestión de usuarios (columna vertebral del proyecto), gestión de los equipos, gestión de categorías, gestión de ligas, gestión de calendario y gestión de clasificación.

Aunque parece poco solo la gestión de usuarios supuso un verdadero avance, una de las principales incorporaciones que supuso el uso de Express JS fue la implementación de JWT para la autenticación de usuarios y la gestión de roles.

Detalle de la API (recomendamos la lectura del archivo README.md dentro del proyecto).

```
api/
├── config/
│   ├── firebase.js      # Configuración de Firebase Admin
├── controllers/
│   ├── usuariosController.js  # Lógica de negocio para usuarios
│   ├── equiposController.js   # Lógica de negocio para equipos
│   ├── categoriasController.js # Lógica de negocio para categorías
│   └── ligasController.js     # Lógica de negocio para ligas
├── middlewares/
└── errorHandler.js      # Manejo centralizado de errores
```

```
| └─ validator.js      # Validación de datos
└─ routes/
    └─ index.js        # Enrutador principal
    └─ usuarios.js     # Rutas de usuarios
    └─ equipos.js      # Rutas de equipos
    └─ categorias.js   # Rutas de categorías
    └─ ligas.js        # Rutas de ligas
```



URL: <https://www.jwt.io>

2.2.6- JWT

JWT (JSON WEB TOKEN) es un sistema de autenticación basado en tokens firmados, nos permiten verificar la identidad de un usuario sin guardar sesiones en el servidor.

El servidor crea un token para el usuario cuando lo autentica y se lo pasa al cliente, este lo enviará religiosamente en cada petición para demostrar quién es. Gracias a este token se puede gestionar los permisos de usuario, no solo a la propia web sino también a la base de datos (lo cual es aún más delicado).

El usuario inicia la sesión en el servidor, en nuestro caso mediante el mail que es el nombre de usuario (lo cogimos porque no admite duplicados en la base de datos), el servidor genera un token con su cabecera, payload (datos del usuario) y signature (firma que garantiza que no ha sido modificado), el cliente lo guarda en su localStorage (en el caso de nuestra web) por último el navegador del cliente envía el token en cada petición, lo cual permite saber exactamente que usuario realiza cada petición verificar su rol y permitirle o denegarle el acceso a ciertas partes de la web.

2.2.7- FRAMEWORKS

Siendo justos hemos decidido dedicarle un apartado a los Frameworks utilizados para el diseño del código en HTML, porque no llegamos a un consenso y cada integrante ha utilizado el que más le ha convenido, según sus necesidades y las tareas que tenía que realizar. En todos los casos el resultado ha sido contrastado en las reuniones del equipo y en las tutorías para verificar su funcionalidad.



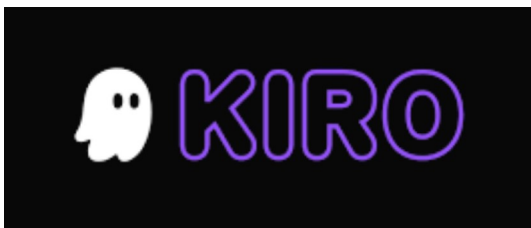
URL: <https://code.visualstudio.com/>

2.2.7.1-VISUALSTUDIO CODE

Uno de los más famosos y utilizados en la actualidad, creado por Microsoft de código gratuito y multiplataforma. Funciona en Windows, macOS y Linux, pensado para cualquier lenguaje de programación. Fue el primer framework que usamos en el proyecto para desarrollar la idea inicial y lo integramos en Git para poder actualizar el repositorio de GITHUB. Dispone de una inteligencia artificial que mediante la introducción de prompts por consola agilizó el trabajo inicial enormemente.

Por desgracia el uso de esta inteligencia artificial inicialmente gratuita no permitió un desarrollo posterior, pero si fue fundamental en los primeros paso y la codificación de la landing inicial de la aplicación. El sistema de depuración fue de una ayuda increíble detectando errores y autocompletando código mientras se escribía.

100% compatible con todas las tecnologías utilizadas en el proyecto (HTML, CSS, JavaScript, Node.js, Firebase, TailwindCSS, etc), mediante la instalación de los paquetes de expansión pudimos crear nuestro primer servidor node.js y hacerlo funcionar en local (fue muy estimulante para todo el equipo la primera vez que funciono).



URL: <https://kiro.dev/>

2.2.7.2-KIRO

Gracias a una recomendación de nuestro tutor Jorge Iván Rodríguez empezamos a familiarizarnos con este entorno de desarrollo, se trata de un micro-framework específico para Node.js, diseñado para crear APIs y aplicaciones web. Muy similar a VisualStudio Code en la estética, es muy potente y también dispone de un chat para comunicarte con la Inteligencia Artificial de la aplicación. El hecho de que este especializado en Node.js nos permitió iniciar una nueva fase en el proyecto. Empezamos a generar la documentación en el propio proyecto mediante la creación de archivos con extensión md.

Hay que decir que en ningún momento hemos dejado de comentar directamente nuestro código, el caso es que al repartirnos las funciones o tareas del proyecto, no podíamos estar llamándonos

constantemente o haciendo reuniones a diario para resolver las dudas que nos iban surgiendo en relación al trabajo de los otros integrantes del equipo. Por ello estos simples archivos, nos permitían comentar las funciones, arquitectura del proyecto, configuración del servidor, comentarios sobre archivos HTML o por ejemplo la migraciones de una API a otra con una gran facilidad, en este punto la IA incorporada por el framework KIRO ha consumido muchos créditos para hacerlo de una forma inteligible y comprensible por una persona.

La inteligencia artificial de KIRO permite su uso a través del consumo de créditos, al mes solo permite 50 créditos, lo cual obliga a utilizarla más en revisión y refactorización de los elementos que en escribir código de forma automatizada. Fue una gran aportación al proyecto.

2.2.8 INTELIGENCIA ARTIFICIAL

En este proyecto se ha utilizado Inteligencia Artificial, nos pusimos el objetivo de utilizar nuevas tecnologías no estudiadas hasta el momento por los integrantes del equipo y hemos necesitado ayuda, no solo de videotutoriales o tutoriales escritos sino también la Inteligencia Artificial nos ha permitido localizar y resolver errores, realizar comentarios y archivos de texto para comprender mejor el proyecto (de haber sido escritos de forma manual hubiera supuesto un trabajo muy denso y largo), la verificación de código y la puesta a punto de muchas funciones de la web.

El primer problema que nos encontramos fue el “sesgo” corporativista de las IA enfrentando y refrendando la información de unas contra otras, por eso buscamos la forma más eficiente de utilizarlas para poder explotarlas en todo su conjunto. A continuación se detallan las IA utilizadas en el proyecto y donde las hemos utilizado:



URL: <https://gemini.google.com/app?hl=es-ES>

2.2.8.1- GEMINI

GEMINI es la inteligencia avanzada de Google, funciona como un asistente conversacional, es la evolución de Google Bard pero con modelos más potentes y muchas más capacidades.

Mediante la introducción de prompts por consola, la IA responde a las preguntas que se formulan o en caso de estar integrada en un Framework puede aumentar la productividad generando código de programación, revisando el código ya existente o ayudando en la configuración del entorno.

Es una herramienta muy potente y muy útil si se utiliza bien. El problema al usar GEMINI, por cierto viene integrado en Firebase, es que tiene un exceso de diligencia y mucha facilidad para “pisar” el código ya escrito, intenta implementar su propio estilo desde el principio y hay que obligarla a trabajar bajo supervisión. Puede destrozar un proyecto en cuestión de segundos.

Esta IA ha sido utilizada sobre todo en el despliegue del proyecto, al ser todo plataforma nativa de Google, GEMINI ha sido muy útil resolviendo gran cantidad de dudas e incluso generando archivos txt y md que hemos utilizado como “chuletas” de la configuración del entorno de desarrollo y ejecución. Desde Firebase se creo la base de datos en Firestore en cuestión de segundos y mediante la Consola de FirebaseStorage se verificaba que los cambios en los registros se hacían de

forma persistente en la base de datos. GEMINI nos ha ahorrado muchas horas de trabajo y estudio para entender como funciona el HOSTING de Firebase y sobre todo nos ha ayudado a resolver los problemas en el despliegue.



URL: <https://chatgpt.com>

2.2.8.2-CHATGPT

ChatGPT es un modelo de inteligencia artificial creado por OPENAI, es un asistente conversacional capaz de responder a preguntas, explicar casi cualquier tema, escribir código y texto y resolver tareas.

Debido a preferencias personales y también a que desde el primer momento no nos conformábamos con la utilización de una única IA por el “sesgo” que pudiera tener y no nos parecía adecuado “jugárnosla a una sola carta”, decidimos incorporarla al proyecto revisando código, funciones, uso de variables y sobre todo para entender la API y su funcionamiento. En este punto esta IA ha sido fundamental, independientemente del código generado en el desarrollo siempre se ha pasado por esta IA como control de calidad para asegurarnos que todo funcionaba bien, sobre todo la API de la web, también ha generado código pero su función básica no ha sido esa. A modo de comentario esta ha sido la única IA de pago que se ha utilizado en el proyecto (porque uno de los integrantes disponía de una licencia en vigor).



URL: <https://copilot.microsoft.com/>

2.2.8.3-COPILOT

Copilot es un asistente de inteligencia artificial creado por Microsoft, diseñado para trabajar más rápido y aprender de forma más profunda. Al igual que otras IA permite la creación de código, la revisión de éste, pero fundamentalmente ha sido útil ya que viene instalada en Windows y permite su utilización como una aplicación nativa en sistemas Windows. Gracias a esa facilidad ha podido explicar el código cuando no se entendía bien y sobre todo ha sido una fuente casi inagotable de conocimiento permitiendo el acceso a explicaciones profundas sobre el funcionamiento de la API, las funciones de ésta, el código JavaScript, la definición de variables, la división de código en bloques y poder revisar código para buscar errores en la programación. La mayor parte de las explicaciones de código las hacíamos entre los integrantes del equipo para no tener que estar

llamándonos diariamente utilizábamos la IA para que nos explicará el código generado por los compañeros, lo cual agilizaba mucho la puesta al día del trabajo realizado por todos.

3. Metodología y desarrollo del proyecto.

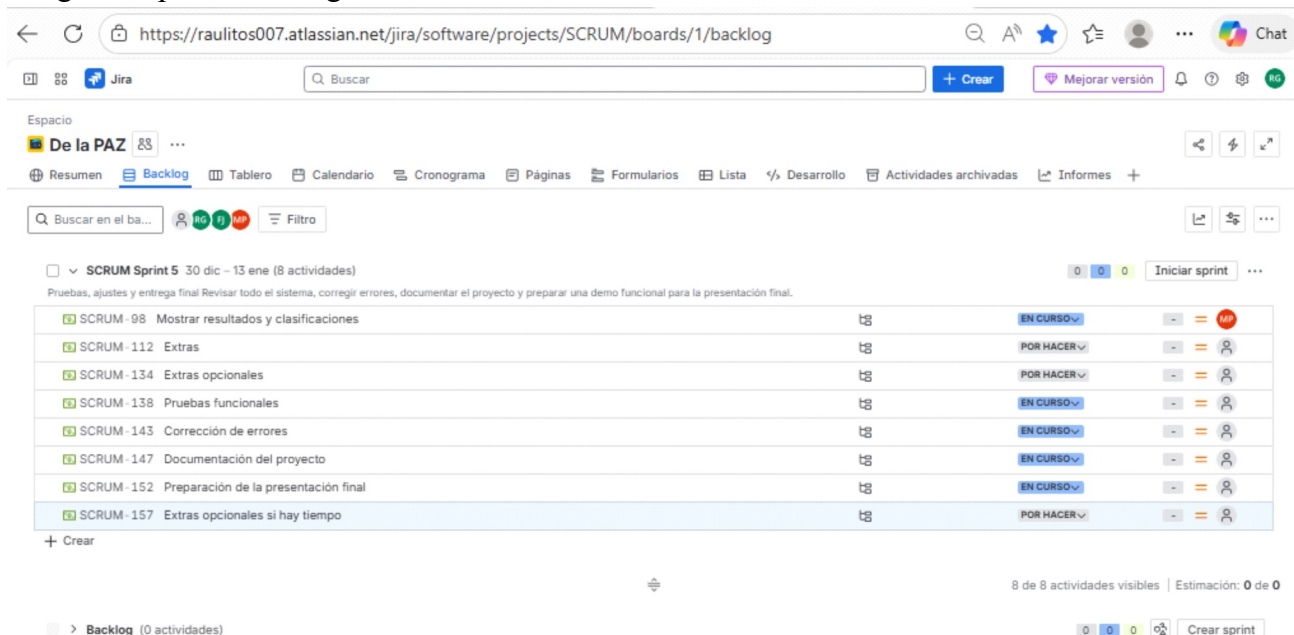
La metodología utilizada en este proyecto ha sido SCRUM, para ello nos hemos apoyado en la herramienta JIRA, para gestión del proyecto creada por Atlassian, es muy utilizada para organizar tareas, seguir incidencias y trabajar con SCRUM y Kanban.



URL: <https://www.atlassian.com>

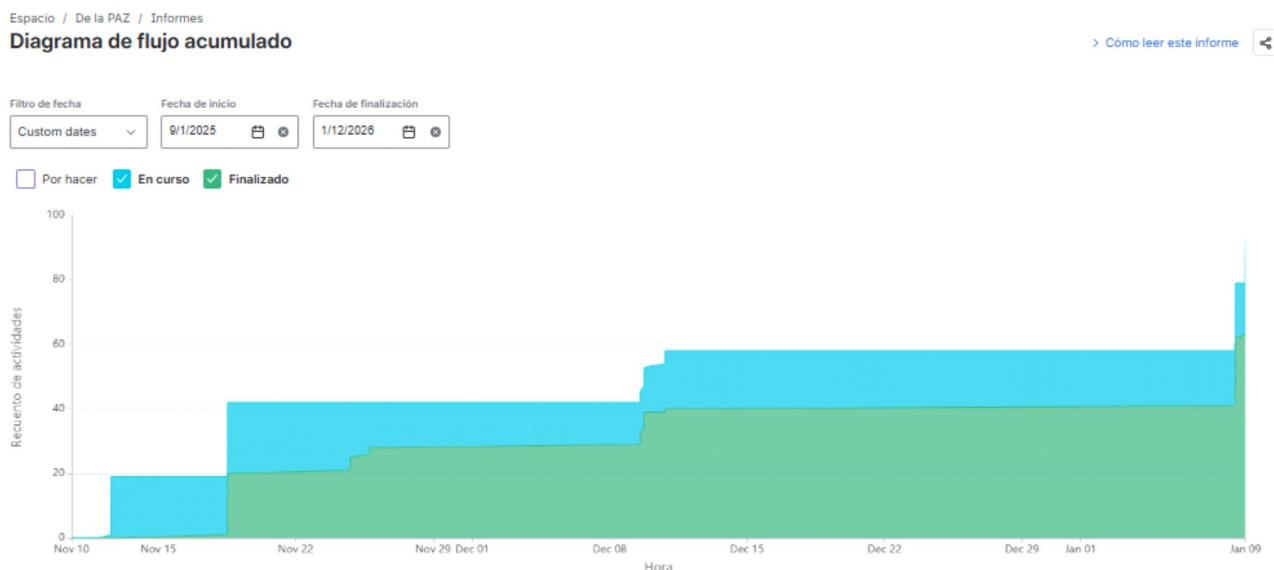
A través de su PANEL y su Dashboard hemos podido organizar las tareas del proyecto, por desgracia hemos priorizado la creación de la web antes que el seguimiento del proyecto a través de la herramienta (el acceso a JIRA ha sido gratuito, tiene una versión de pago que no ha sido utilizada en el proyecto).

Imagen del panel Backlog de JIRA:



Cada dos semanas más o menos se hace un Sprint, el proyecto entero fue dividido en 5 Sprint, el Sprint 1 fue la primera tutoría de proyecto que tuvimos con nuestro tutor. Desde hay se han ido sucediendo cada dos semanas más o menos coincidiendo con las reuniones de seguimiento de las tutorías. El último Sprint 5, quedaba fuera de plazo, en la imagen se pueden visualizar algunas tareas incluidas en dicho Sprint. Lo bueno de esta aplicación es que si una tarea no esta concluida te la incluye en el siguiente Sprint de forma automática, genera informes para valorar el desempeño y la rapidez con la que se desarrolla el proyecto, permite poner interrupciones si el proyecto no avanza y mandar mensajes a los integrantes, así como cambiar las tareas o el autor de las mismas.

Dado que no ha sido actualizado diariamente como debería, el único informe que podemos aportar es este diagrama de flujo acumulado:



Nos ha parecido una herramienta muy potente, pero necesita un mantenimiento diario y no hemos podido atenderla como se merecía.

3.1 Historias de usuario

Dentro de la metodología SCRUM es muy habitual incluir o mejor dicho trabajar en base a lo que se denomina historias de usuario.

En nuestro caso concreto los usuarios son dados de alta de forma centralizada, es decir, a través de la sección contacto o quiero registrarme se ponen en contacto con los administradores y son ellos los que proceden a cursar su solicitud, ningún usuario puede directamente darse de alta en la web, debe recurrir a un administrador para hacerlo.

USUARIO ANÓNIMO

El usuario anónimo solo puede acceder a la landing y a los enlaces que se encuentran en ella.

No puede modificar datos de ningún tipo y solo puede acceder a los datos denominados públicos para poder revisar los resultados, equipos incluidos en las ligas, consultar las ligas y por supuesto los próximos encuentros. En ningún caso puede acceder al perfil de ningún usuario, ni de ningún jugador.

No puede logearse porque no dispone de usuario y contraseña.

ADMINISTRADORES

A priori cabría pensar que el control que tienen los administradores sobre el sitio es total, pero no es así. El administrador puede hacer muchas cosas, pero no puede, borrar la base de datos, crear colecciones nuevas y de hecho no puede acceder a la colección JUGADORES donde se encuentran las fichas privadas de los jugadores inscritos en los equipos. A partir de ahí puede crear, modificar y

borrar usuarios, crear categorías, ligas y calendarios para las mismas.

NO PUEDEN: acceder a la colección de JUGADORES, las funciones propias para borrar la base de datos de forma total o la colección dentro de la base de datos NO ESTÁN IMPLEMENTADAS en la API del sitio web, para hacer eso debería estar logeado como uno de los desarrolladores habilitados en el proyecto cuyos logins no coinciden con los de los administradores del sitio.

DIRECTORES DE EQUIPOS Y DELEGADOS DE CAMPO

Los directores de equipo o delegados de campo tienen una función delegada muy grande, son los encargados del registro de los jugadores en sus propios equipos, deben acceder solo a su equipo, en el caso de que el equipo tenga varias categorías debe poder trabajar con todas ellas.

El control que deben tener es total sobre los datos de sus equipos, deben poder borrar un jugador, cambiar los datos de jugadores en caso de ser necesario, incluso cambiarlos de categoría dentro de un mismo equipo, pero no pueden cambiar un jugador a otro equipo que no sea el suyo.

Esa funcionalidad no esta incluida en la API, habría que incluirla y sería un rol que habría que desarrollar.

NO PUEDEN: borrar sus propios equipos o cualquier otro equipo, no pueden crear nuevas categorías, no pueden borrar ligas, incluir sus equipos en las ligas, modificar usuarios o datos de ningún equipo.

ENTRENADORES

Los entrenadores pueden acceder a la ficha de sus jugadores y consultar su estado (activo, sancionado, lesionado), para saber que jugadores tiene disponibles para cada partido. Partimos de la base de que un jugador sin estar registrado en el equipo no puede ni debe jugar, si el jugador está registrado en una categoría distinta a la suya tampoco, gracias al filtro de DNI nos aseguramos que por un lado los DNIs introducidos en los registros de jugadores sean validos y a la vez no exista otro registro con el mismo DNI para evitar duplicados. Un entrenador también puede acceder al calendario de las ligas y al calendario del equipo que entrena de forma personalizada. Un entrenador solo entrena un equipo de una categoría, en otra funcionalidad se podría permitir que un entrenador entrenase dos equipos pero habría que dotar a la API de una lógica que impidiera que dos equipos de un mismo entrenador se enfrentasen entre ellos o que un entrenador tuviera dos partidos al mismo tiempo en campos diferentes el mismo día y a la misma hora. Lo que si se permite es que haya varios entrenadores en un mismo equipo y categoría.

NO PUEDEN: modificar datos de los jugadores, modificar datos de ningún equipo, acceder a la base de datos de USUARIOS de la aplicación, crear ligas, generar calendarios ni cambiarlos. Después del rol ANONIMO es el rol con menos permisos.

ÁRBITROS Y COMITÉS DISCIPLINARIOS

Su labor es registrar las actas de los partidos que arbitren, también pueden cambiar el arbitro de otros encuentros siempre y cuando sea para asignarselo ellos mismos, no pueden asignarle PARTIDOS a otros arbitros. El sorteo de equipos y arbitros se realiza automáticamente al realizar los emparejamientos de los equipos se les asigna un arbitro de forma totalmente aleatoria (con Round Robin). Solo se registra el número ID de USUARIO del colegiado asignado, gracias a este dato se pueden extraer todos los datos del colegiado que registrará el acta del partido, este acta es un

registro de campos dentro del propio registro del Partido, donde a parte de los equipos LOCAL Y VISITANTE, se disponen otros campos como tarjetas amarillas, rojas, corners y por supuesto goles local y goles visitante para poder extraer los resultados de los partidos, también puede cambiar el estado de un jugador a SANCIONADO. Con respecto a las actas se podrían implementar varias cosas, por ejemplo acumulación de tarjetas o suspensión del jugador para toda la liga.

NO PUEDEN: modificar datos de USUARIOS, crear ligas, crear categorías, generar calendarios, borrar jugadores o incluir jugadores en un equipo.

Estos son los roles, dentro de JIRA están expresados con otras palabras, las historias de usuario suelen empezar con un “yo como usuario con el rol ...”, lo cual permite ponerse en la piel del usuario en cuestión y ver cuales son las necesidades para realizar sus tareas.

3.2 Plan de Proyecto.

Este es el resumen de Sprint, tareas y subtareas del proyecto.

Fue diseñado como un plan de proyecto para JIRA.

Sprint 1 -Inicial Reunión Tutoría Scrum 1

Estructura del proyecto

- Crear proyecto en base con node.js.

- Configurar entorno local y repositorio GitHub

- Instalar dependencias necesarias (Firebase, Dotenv, etc)

- Diseño Base de Datos

Navegación y diseño inicialmente

- Crear barra de navegación con enlaces a Inicio, Ligas, Clasificaciones, Estadísticas, Contacto.

- Diseñar página de inicio con logo, descripción y bienvenida

- Definir colores, tipografía y estilo general (CSS o framework como TailwindCSS)

Conexión con Firestore

- Configurar Firebase en el proyectos

- Crear archivo de conexión a Firestore

- Probar lectura y escritura básica

Extras

- Preparar Demo con navegación funcional y conexión a Firestore

- Reunión de revisión con el equipo

- Recoger feedback y preparar backlog para Sprint 2.

Sprint 2 – 13 al 21 noviembre 2025

Crear ligas desde interfaz

- Diseñar formulario para crear una liga (nombre, categoría, fecha de inicio)
- Validar campos del formulario (obligatorios, formato correcto)
- Guardar liga en Firestore dentro de la colección ligas
- Mostrar mensaje de éxito o error

Crear categorías asociadas a ligas

- Diseñar selector o formulario para añadir categorías (nombre. Edad mínima y máxima)
- Guardar categoría en Firestore como subcolección o campo dentro de ligas
- Mostrar categorías en vista de liga

Mostrar ligas y categorías en la web

- Crear vista de ligas con tarjetas o lista
- Mostrar categorías dentro de cada liga
- Conectar con Firestore par leer datos en tiempo real

Pruebas y revisión interna

- Probar creación de ligas y categorías con datos reales
- Revisar estricta en Firestore y corregir si es necesario
- Reunión de revisión y feedback del equipo

Back-up del proyecto

Extras

- Añadir opción para editar o eliminar ligas/categorías
- Mostrar número de equipos por categoría
- Preparar documentación de estructura de Firestore actualizada

Sprint 3 – 30 noviembre al 14 de diciembre de 2025

Mostrar calendario de partidos

- Crear vista de calendario con fecha, hora y lugar.
- Leer datos desde Firestore (colección partidos)
- Ordenar partidos por fecha y categoría
- Mostrar estado del partido (pendiente, jugado y cancelado)

Mostrar resultados y clasificaciones

- Crear tabla de clasificación por liga
- Calcular puntos, goles a favor/en contra, partidos jugados
- Mostrar resultados recientes por jornada
- Leer y actualizar datos en Firestore (resultados, clasificaciones)

Pruebas y revisión interna

- Probar visualización con datos reales
- Validar cálculos de clasificación y estadísticas
- Reunión de revisión y feedback del equipo

Autenticación de usuarios

- Configurar autenticación (email/password)
- Crear formulario de login con validación
- Mostrar mensajes de error (usuario no existe, contraseña incorrecta)
- Redirigir al panel tras login exitoso

Roles de usuario

- Definir roles en Firestore (usuarios con campo rol)
- Mostrar contenido diferente según el rol
- Proteger rutas del panel para que solo accedan administradores

Información cookies y política de privacidad

- Crear fichero política de cookies
- Crear fichero privacidad
- Enlazar ficheros con el footer

Sprint 4 – 15 al 29 de diciembre de 2025

(se pasaron Roles de usuario y autenticación del sprint 4 al 3 por recomendación del tutor por eso este Sprint es más corto en actividades)

Extraes

- Añadir gráficos simples (barras o líneas) para estadísticas
- Mostrar partidos destacados en la página de inicio
- Exportar estadísticas a PDF o CSV (solo si lo necesitan)

Pruebas y revisión interna

- Probar login con usuarios reales
- Validar que los visitantes no accedan al panel
- Reunión de revisión y feedback del equipo

Extras opcionales

- Añadir recuperación de contraseña
- Mostrar el nombre del usuario en el panel
- Añadir confirmación antes de eliminar datos

Sprint 5 – 30 de diciembre de 2025 – 13 de enero de 2026

Pruebas Funcionales

- Probar navegación completa (menú, enlaces, vistas)
- Probar creación y visualización de ligas, partidos y estadísticas)
- Probar login y acceso al panel de administración
- Validar que los datos se guardan y leen correctamente desde Firestore

Corrección de errores

- Revisar errores en consola y corregirlos
- Ajustar estilos y diseño para que sea responsivo
- Corregir errores en formularios y validaciones

Documentación del proyecto

- Crear documento con estructura de Firestore (colecciones y campos)
- Documentar rutas y vistas del frontend
- Explicar cómo desplegar el proyecto (Firebase Hosting, etc)
- Incluir roles y funciones del equipo

Preparación de la presentación final

- Crear guion de presentación (quién habla, qué se muestra)
- Preparar entorno de demo con datos reales
- Ensayar presentación con el equipo
- Crear diapositivas si es necesario

Extras opcionales si hay tiempo

- Añadir animaciones o mejoras visuales

- Crear versión móvil optimizada

- Añadir sección de preguntas frecuentes (FAQ)

Los roles dentro del proyecto los repartimos de la siguiente manera.

Diseño visual y maquetación: Francisco José Tieso Muñoz.

Funciones: Definir la identidad visual del proyecto (colores, tipografías, estilos), crear mockups de las pantalla principales, maquetas de vistas usando HTML y CSS.

Diseño API: María Parreño Castillejo.

Funciones: Diseñar la estructura de la API (endpoints, modelos de datos, flujos), implementar la lógica del backend con el lenguaje, gestionar la comunicación entre frontend y backend, asegurar validaciones, manejo de errores e incorporar la seguridad en la autenticación de la web.

Implementación y despliegue: Raúl García Gómez

Funciones: Configurar el entorno de despliegue (hosting, base de datos, variables de control), automatizar documentos del proceso de despliegue, supervisar el funcionamiento y realizar pruebas de uso para aplicar correcciones y asegurar la disponibilidad del proyecto.

3.3 Arquitectura de la aplicación. Esquema de los componentes con tecnologías asociadas

Nuestro proyecto es una aplicación web deportiva que se basa en archivos HTML con JavaScript incrustado bien directamente en el propio archivo o bien incrustado mediante funciones importadas de otros archivos del proyecto para crear las peticiones al servidor. Además dispone de su propia API REST para hacer peticiones a la base de datos en este caso Firestore.

En el siguiente esquema mostramos de forma gráfica como trabaja:

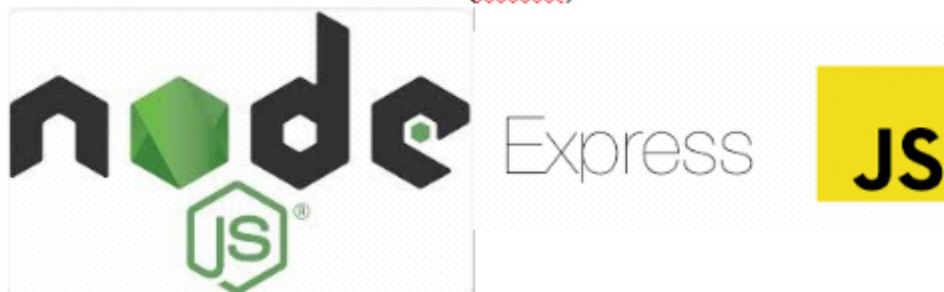
USUARIO FINAL (NAVEGADOR WEB)



CAPA FRONTEND



CAPA API (Backend)



HTTP REQUEST

MIDDLEWARES

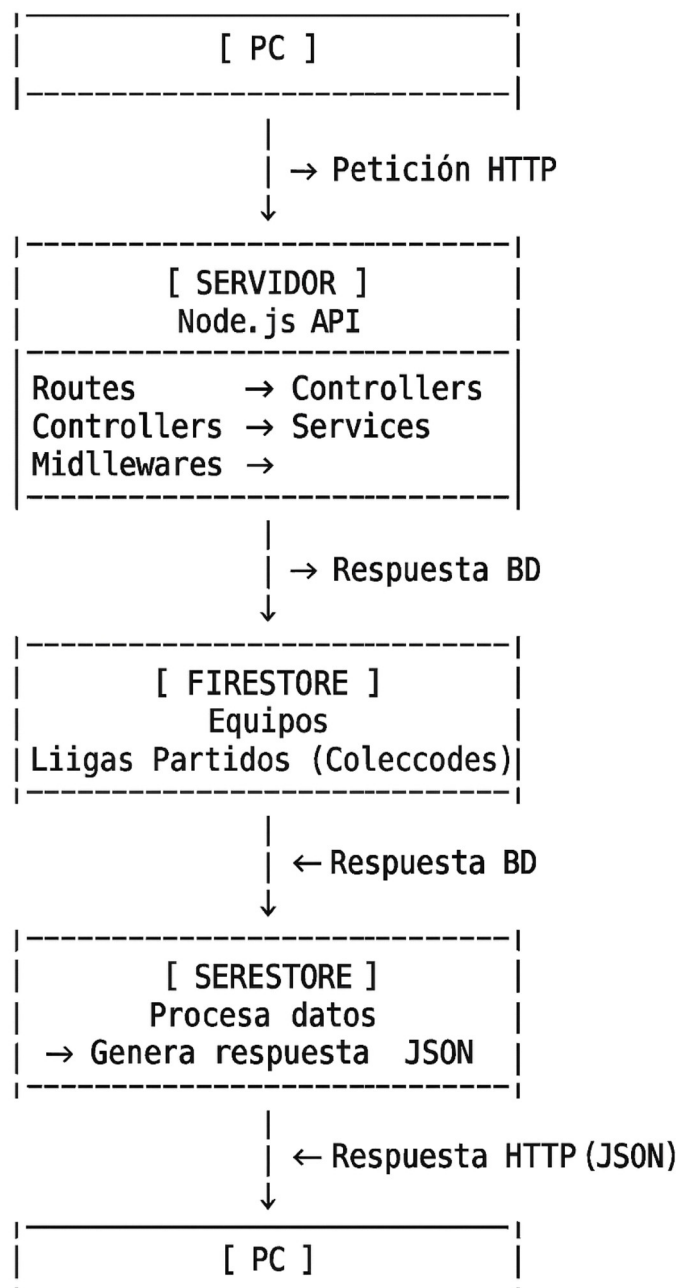
Autenticación, validación y manejo de errores



El flujo de una petición estándar de un usuario sería así:

- El usuario utiliza la vista HTML
- JS incrustado llama a las funciones de la API.
- La API procesa y responde, enviando una petición HTTP al servidor.
- Se procesa la petición en la API del servidor Backend
- El servicio consulta a la base de datos mediante SDK
- La base de datos devuelve los datos al servidores
- El servidor procesa la petición y la devuelve al cliente FrontEnd
- Es el FrontEnd quien actualiza la vista.

De una forma más clásica:



3.4 Diseño de la aplicación.

El diseño de la aplicación es muy simple, se utiliza la landing (index.html) como página principal, tiene una parte que es pública compuesta por los botones de Ver Clasificaciones y Próximos encuentros, y una serie de enlaces presentes en el encabezado (común a todas las páginas), las secciones de Equipos, Categorías, Ligas, Clasificaciones, Resultados, Calendario y Próximos encuentros permiten acceder de forma pública a los resultados de las diferentes ligas y categorías así como estar informado de los próximos encuentros. Por otro lado el botón de LOGIN permite a los usuarios registrados por la asociación desarrollar uno de sus cuatro roles:

ADMINISTRADORES

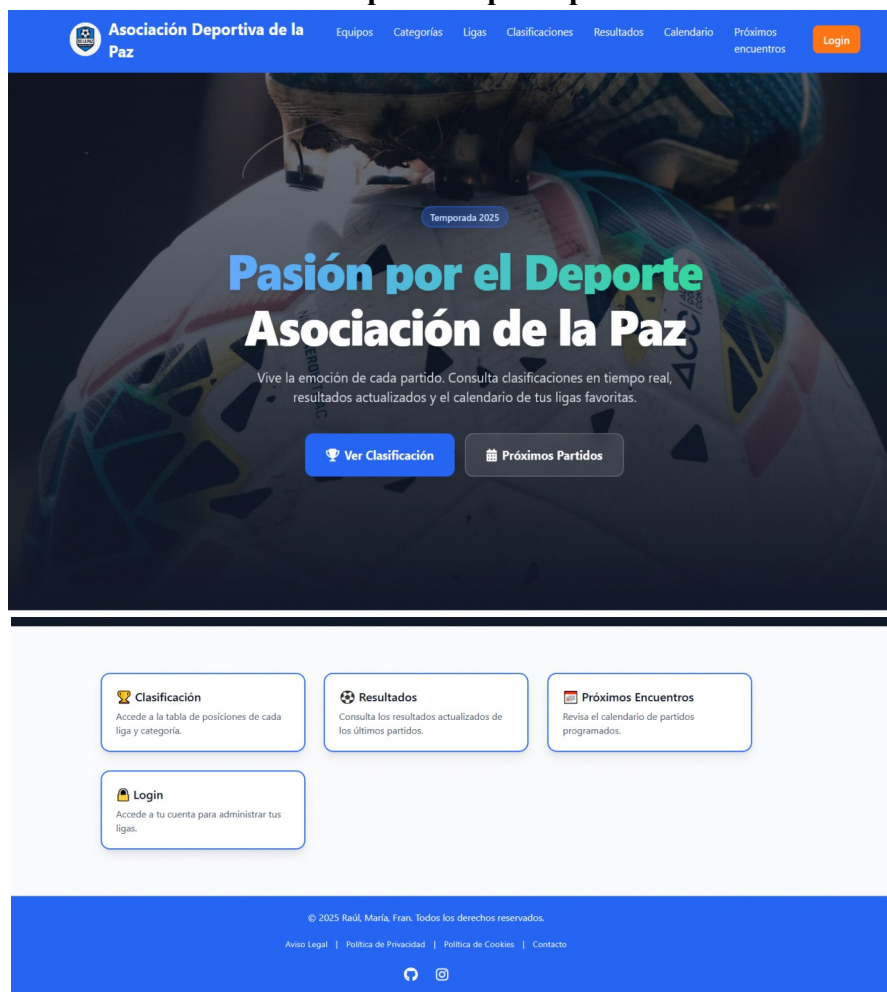
DELEGADOS DE CAMPO O DIRECTORES DE EQUIPO

ENTRENADORES

ÁRBITROS

Cada uno de estos roles tiene una serie de funciones que le son propias y no pueden ser realizadas por el resto, el Entrenador es el perfil más bajo, dado que no puede modificar nada solo consultar los datos de “su equipo”.

Vista pantalla principal:



Vista panel Delegado de Campo o Director Deportivo

**Asociación Deportiva de la Paz**

[Equipos](#) [Categorías](#) [Ligas](#) [Clasificaciones](#) [Resultados](#) [Calendario](#) [PANEL](#) [Cerrar Sesión](#)

 Volver

**Paco Porras**
ID: Cy8TnjWtbGEglM7bm32TU32fDwi2

**Delegado de: DREAM TEAM**

**Mis Categorías**
Gestiona las categorías y jugadores de tu equipo.
[Ver Categorías →](#)

**Gestionar Jugadores**
Edita y gestiona todos los jugadores de tu equipo.
[Gestionar →](#)

**Horarios**
Verifica horarios y campos de juego de tu equipo.
[Consultar →](#)

© 2025 Raúl, María, Fran. Todos los derechos reservados.

[Aviso Legal](#) | [Política de Privacidad](#) | [Política de Cookies](#) | [Contacto](#)

Vista panel Entrenador

**Asociación Deportiva de la Paz**

[Equipos](#) [Categorías](#) [Ligas](#) [Clasificaciones](#) [Resultados](#) [Calendario](#) [PANEL](#) [Cerrar Sesión](#)

 Volver

**Antonio Banderas**
ID: ETytFxo8tbT0lmuWAojoNuwnF22

**Mi Equipo: FOREVER YOUNG**

**Mi Plantilla**
Gestiona los jugadores, dorsales y posiciones de tu equipo.
[Ver Jugadores →](#)

**Mis Partidos**
Calendario y resultados solo de tu equipo.
[Ver Calendario →](#)

**Mi Posición**
Posición de tu equipo en la clasificación.
[Ver Posición →](#)

© 2025 Raúl, María, Fran. Todos los derechos reservados.

[Aviso Legal](#) | [Política de Privacidad](#) | [Política de Cookies](#) | [Contacto](#)

Fecha: 11/01/2026

26 de 55

Vista Panel de Arbitro

Asociación Deportiva de la Paz

Equipos

Categorías

Ligas

Clasificaciones

Resultados

Calendario

PANEL

Cerrar Sesión

Panel de Árbitro

Gestión de actas, resultados y designaciones.

Ana Del Olmo

ID: v5ya82Kktw0eHW7Yt49D

Mis Designaciones

Consulta los partidos que te han sido asignados.

Ver Partidos →

Normativa

Consulta el reglamento y códigos de conducta.

Leer Reglamento ↗

© 2025 Raúl, María, Fran. Todos los derechos reservados.

Aviso Legal

Política de Privacidad

Política de Cookies

Contacto

Vistas públicas
EQUIPOS

Asociación Deportiva de la Paz

Equipos

Categorías

Ligas

Clasificaciones

Resultados

Calendario

Próximos encuentros

Login

Listado Equipos Consulta

Consulta la información de todos los equipos registrados

← Volver

Buscar equipo

Escribe el nombre del equipo...

Filtrar por categoría

Todas las categorías ▾

Filtrar por tipo

Todos los tipos ▾

✕ Limpiar

↻ Actualizar

📊 Total equipos: 24

🟢 Mostrando: 24

🔴 Filtros activos: 0

NOMBRE DEL EQUIPO ▴ ▾	CATEGORÍA ▴ ▾	TIPO ▴ ▾
L Los Resaka Equipo registrado	Veteranos	MASCULINO
R REGUETONEROS S.XXI Equipo registrado	Prebenjamin	MASCULINO
E ESTUDIANTES Equipo registrado	Prebenjamin	FEMENINO
U Unión Juvenil San Isidro Equipo registrado	Juvenil	MASCULINO
T TRAGALDABAS Equipo registrado	Alevín	MASCULINO

Fecha: 11/01/2026

27 de 55

CATEGORÍAS

**Asociación Deportiva de la Paz**

[Equipos](#)[Categorías](#)[Ligas](#)[Clasificaciones](#)[Resultados](#)[Calendario](#)[Próximos encuentros](#)[Login](#)

Listado Categorías Consulta

Consulta la información de todas las categorías registradas

Buscar categoría

Escribe el nombre de la categoría...

Filtrar por tipo

Todos los tipos

Edad mínima

Ej: 6

Edad máxima

Ej: 12

X Limpiar

Actualizar

Total categorías: 11

Mostrando: 11

Filtros activos: 0

NOMBRE	RANGO DE EDADES	AÑOS NACIMIENTO	TIPO	DENOMINACIÓN
I Infantil Categoría deportiva	12 - 13 años	2013 - 2012	MASCULINO	Sub-14
S Sin nombre Categoría deportiva	5 - 6 años	2020 - 2019	MIXTO	Promesas
B Benjamin Categoría deportiva	8 - 9 años	2015 - 2014	FEMENINO	Sub-10
J Juvenil Categoría deportiva	16 - 18 años	2009 - 2007	MASCULINO	Sub-19
A Alevín Categoría deportiva	10 - 11 años	2015 - 2014	MASCULINO	Sub-12

LIGAS

**Asociación Deportiva de la Paz**

[Equipos](#)[Categorías](#)[Ligas](#)[Clasificaciones](#)[Resultados](#)[Calendario](#)[Próximos encuentros](#)[Login](#)

Listado Ligas Consulta

Consulta la información de todas las ligas registradas

Buscar liga

Escribe el nombre de la liga...

Filtrar por temporada

Todas las temporadas

Filtrar por categoría

Todas las categorías

Filtrar por tipo

Todos los tipos

X Limpiar

Actualizar

Total ligas: 9

Mostrando: 9

Filtros activos: 0

LIGA	TEMPORADA	CATEGORÍA	EQUIPOS	ESTADO CALENDARIO	ACCIONES
Juventudes Asociadas MASCULINO	2025-2026	Juvenil	10 equipos	X NO GENERADO	Equipos
Pruebas MASCULINO	2024-2026	Juvenil	5 equipos	✓ GENERADO	Equipos Ver Calendario
Perturbados MASCULINO	2025-2026	Veteranos	4 equipos	✓ GENERADO	Equipos Ver Calendario
los nuevines FEMENINO	2026	Benjamin	0 equipos	X NO GENERADO	Equipos
Liga infantil Virgen de la PAZ MASCULINO	2025-2026	Infantil	0 equipos	X NO GENERADO	Equipos

CLASIFICACIONES

Asociación Deportiva de la Paz

Equipos

Categorías

Ligas

Clasificaciones

Resultados

Calendario

Próximos encuentros

Login

Tabla de Clasificación

Sigue la trayectoria de tu equipo jornada a jornada

Volver

Seleccionar Liga

Pruebas

Ver Clasificación

POS	EQUIPO	PJ	PG	PE	PP	GF	GC	DG	PTS
1	U Unión Juvenil San Isidro 15 puntos	8	4	3	1	19	13	+6	15
2	S Sociedad Deportiva El Encinar 14 puntos	8	4	2	2	26	18	+8	14
3	C Club Deportivo Nueva Castilla 11 puntos	8	3	2	3	13	16	-3	11
4	U Unión Deportiva Las Águilas 8 puntos	8	2	2	4	17	23	-6	8
5	C Club de Fútbol Villa del Río 6 puntos	8	1	3	4	13	18	-5	6

Última actualización: Hoy

RESULTADOS

Asociación Deportiva de la Paz

Equipos

Categorías

Ligas

Clasificaciones

Resultados

Calendario

Próximos encuentros

Login

Resultados de Partidos

Consulta los resultados actualizados de los últimos partidos disputados.

Volver

Visualizador de Liga

Selecciona una liga para visualizar los partidos.

Liga

Pruebas

Ver Liga

Partidos de Pruebas (20 partidos)

viernes, 26 de enero de 2024

07:03

Unión Juvenil San Isidro

0 - 0

Unión Deportiva Las Águilas

Jornada 4

PABELLÓN (Cubierto)

Árbitro: 9yHYVYG15gFN5Xl3bst6

finalizado

viernes, 19 de enero de 2024

07:01

Unión Juvenil San Isidro

0 - 0

Club de Fútbol Villa del Río

Jornada 3

PABELLÓN (Cubierto)

Árbitro: JAF0pbpNd3KGSE0rpGL8

finalizado

viernes, 23 de febrero de 2024

07:08

Club Deportivo Nueva Castilla

3 - 2

Unión Deportiva Las Águilas

Jornada 8

PABELLÓN (Cubierto)

Árbitro: 9yHYVYG15gFN5Xl3bst6

finalizado

sábado, 3 de febrero de 2024

07:03

Club Deportivo Nueva Castilla

2 - 2

Club de Fútbol Villa del Río

Jornada 5

PABELLÓN (Cubierto)

Árbitro: Qu4YVVG15nFN5Xl3bst6

viernes, 16 de febrero de 2024

07:05

Club Deportivo Nueva Castilla

2 - 3

Unión Juvenil San Isidro

Jornada 7

PABELLÓN (Cubierto)

Árbitro: lAF0pbnNd3KGSE0rmGL8

sábado, 13 de enero de 2024

07:00

Unión Juvenil San Isidro

2 - 3

Club Deportivo Nueva Castilla

Jornada 2

PABELLÓN (Cubierto)

Árbitro: Qu4YVVG15nFN5Xl3bst6

Fecha: 11/01/2026

29 de 55

CALENDARIO

Asociación Deportiva de la Paz

Equipos

Categorías

Ligas

Clasificaciones

Resultados

Calendario

Próximos encuentros

Login

Calendario de Liga

Consulta las jornadas y partidos programados

←

Volver

Seleccionar Liga

Pruebas (5 equipos)

Ver Calendario

Pruebas

Temporada:
2024-2026

Categoría:
Juvenil

Tipo:
MASCULINO

Equipos:
5 equipos

Información de Liga

10

Jornadas

20

Partidos

20

Jugados

0

Pendientes

Jornada 1

Unión Juvenil San Isidro descansa

2 partidos

Club Deportivo Nueva Castilla

0 - 0

Sociedad Deportiva El Encinar

LOCAL

VISITANTE

viernes, 12 de enero de 2024

07:00

PABELLÓN (Cubierto)

Árbitro: v5us82YkhuDeHW7Vn4RD

Club de Fútbol Villa del Río

1 - 2

Unión Deportiva Las Águilas

LOCAL

VISITANTE

viernes, 12 de enero de 2024

07:00

PABELLÓN (Cubierto)

Árbitro: 7RS5mSdG4n5CzDrDx5Wlk

PRÓXIMOS ENCUENTROS

Asociación Deportiva de la Paz

Equipos

Categorías

Ligas

Clasificaciones

Resultados

Calendario

Próximos encuentros

Login

Próximos Encuentros

Partidos de los últimos 7 días y próximos 7 días (programados, suspendidos y finalizados).

←

Volver

Filtrar por Liga

Visión General (Por Días)

Próximos

Viernes, 9 de enero de 2026

Liga Cadete de la Asociación de la Paz

06:54

PROGRAMADO

D

VS

L

DREAM TEAM

Jornada 2

Las ganadoras

LAS TULIPAS II

Ver Detalles >

Sábado, 10 de enero de 2026

Liga Cadete de la Asociación de la Paz

06:53

Perturbados

10:00

Liga Test Bearer Prebenjamin

10:00

Fecha: 11/01/2026

30 de 55

Dependiendo del rol se pueden hacer unas cosas u otras:

Administradores no son los desarrolladores del sitio, ese perfil no esta contemplado, son los gestores de la asociación y pueden:

- Crear, editar y borrar ligas (CRUD).
- Crear, editar y borrar categoría (CRUD).
- Crear, editar y borrar equipos (CRUD).
- Crear, editar y borrar usuarios (CRUD).
- Crear, editar y borrar partidos (CRUD).
- Una de las últimas incorporaciones en la base de datos MENSAJERÍA, funciona como un mail pero no lo es, los usuarios a través del enlace público Contacto le pueden dejar mensajes que solo verán los administradores del sitio.

Vista de CONTACTO

**Asociación Deportiva de la Paz**

EquiposCategoríasLigasClasificacionesResultadosCalendarioPróximos encuentros

Login

Contacto y Solicitudes

¿Necesitas registrar un nuevo usuario o tienes alguna duda? Envíanos tu solicitud aquí.

Nombre Completo
 Tu nombre

Correo Electrónico
 tucorreo@ejemplo.com

Asunto
 Selecciona un motivo... ▾

Mensaje
Escribe aquí los detalles de tu solicitud...

☐ He leído y acepto la [política de cookies](#)

Enviar Solicitud

Nota: solo si se aceptan las cookies se puede enviar el formulario, leerlas no es obligatorio y están accesibles así como la Política de Privacidad y el Aviso legal en todas las páginas en el footer.

Vista del menú mensajes del panel del administrador.

Gestión de Mensajes
Administra los mensajes de contacto y solicitudes recibidas. [← Volver al Panel](#)

Total Mensajes: **2**
No Leídos: **0**
Leídos: **2**

Filtros de Búsqueda

Buscar por nombre:
Buscar por email:
Filtrar por asunto: Todos los asuntos
Estado de lectura: Todos

[Buscar](#) [Limpiar](#)

Lista de Mensajes

Roberto Soporte Técnico Leído Eliminar
✉ robertito@gmail.com
📅 Invalid Date

Mensaje:
Fallo al conectarme al servicio

Al ser una base de datos NOSQL no tenemos problema con las posibles inyecciones de código SQL en el formulario, debido a que si las consultas al resto de bases de datos no se realizan a través de la API del proyecto el servidor no las escucha, si que se podría insertar código JavaScript y HTML.

No nos extendernos en todas las funciones de todos los usuarios pero los roles determinan la capacidad de acceso a la API esta mediante JWT verifica la identidad del usuario y responde en consecuencia a la petición.

Delegados de campo y Directores deportivos:

- Crear, editar y borrar Jugadores (CRUD), solo de los inscritos en sus equipos.

Entrenadores:

- Solo pueden acceder a los registros de los jugadores de sus equipos, no pueden modificar datos, son un observador privilegiado porque solo ven los datos de sus jugadores.

Árbitros:

- Pueden acceder a los registros de PARTIDOS, pero solo a unos pocos campos, no pueden modificar la fecha ni la hora del encuentro, su aporte es fundamental porque son los que introducen los resultados de los partidos, ningún otro usuario puede hacerlo, sólo de los partidos que le han sido asignados.

Vista acceso al acta de un arbitro asignado.



Ana Del Olmo
Árbitro Principal

14
Partidos asignados

Filtros:

Todas las ligas

Todos los estados

Ordenar por fecha

Actualizar

Limpiar


REGUETONEROS S.XXI vs PELOTEROS
Liga Test Bearer Prebenjamin


sábado, 10 de enero de 2026


10:30


LAS TULIPAS


Programado


Acta


Jornada 1

Vista del acta del partido

Acta del Partido
REGUETONEROS S.XXI vs PELOTEROS - Liga Test Bearer Prebenjamin

Guardar Cambios

Volver


Información del Partido

Liga (Solo lectura)

Liga Test Bearer Prebenjamin

Jornada (Solo lectura)

1

Fecha (Solo lectura)

01/10/2026, 10:30 AM

Campo (Solo lectura)

LAS TULIPAS

Tiempo de Juego (min)

0

Estado

Programado


REGUETONEROS S.XXI

Goles

0

Tarjetas Amarillas

0

Tarjetas Rojas

0

Faltas

0

Corners

0


PELOTEROS

Goles

0

Tarjetas Amarillas

0

Tarjetas Rojas

0

Faltas

0

Corners

0

En relación con los aspectos gráficos lo desarrollamos en el Manual de Identidad Visual de la Web.

3.4.1 Diseño “Frontend”.

Consideramos que una imagen vale más que mil palabras se puede resumir con este esquema:



Mediante el uso de JavaScript incrustado y la ayuda de las librerías TailwindCSS , hemos conseguido por un lado tener una consistencia en el diseño y la funcionalidad de las webs dado que aporta un diseño responsivo adaptativo al tamaño de la pantalla del dispositivo que donde se visualiza, además de tener una mejora considerable en el diseño que la hace atractiva al usuario. Nuestra paleta de colores y estilo se encuentra en el ANEXO A Manual de identidad visual del sitio.

Disponemos de un header en todas las páginas:

```
<!-- Estructura estándar del header -->
<div class="flex flex-col md:flex-row justify-between items-start md:items-center mb-8 gap-4">
  <div>
    <h1 class="text-4xl font-extrabold text-primary mb-2">
      <i class="fas fa-[icono] mr-3"></i>[Título]
    </h1>
    <p class="text-lg text-gray-600">[Descripción]</p>
  </div>
  <div class="flex gap-3">
    <button class="bg-gray-500 hover:bg-gray-600 text-white py-3 px-6 rounded-xl">
      <i class="fas fa-arrow-left"></i>Volver
    </button>
    <!-- Botones adicionales según la página -->
  </div>
</div>
```

Tarjetas de Filtros

```
<!-- Estructura de filtros -->
<div class="bg-white p-6 rounded-2xl shadow-lg border-2 border-primary mb-6">
  <div class="flex flex-col lg:flex-row gap-4 items-end">
    <!-- Campos de filtro -->
    <div class="flex-1">
      <label class="block text-gray-700 font-medium mb-2">Campo</label>
      <input class="w-full px-4 py-2 border rounded-lg focus:ring-2 focus:ring-primary">
    </div>
    <!-- Botones de acción -->
    <div class="flex gap-2">
      <button class="bg-primary hover:bg-blue-700 text-white px-4 py-2 rounded-lg">
        Filtrar
      </button>
    </div>
  </div>
</div>
```

Y estados de carga

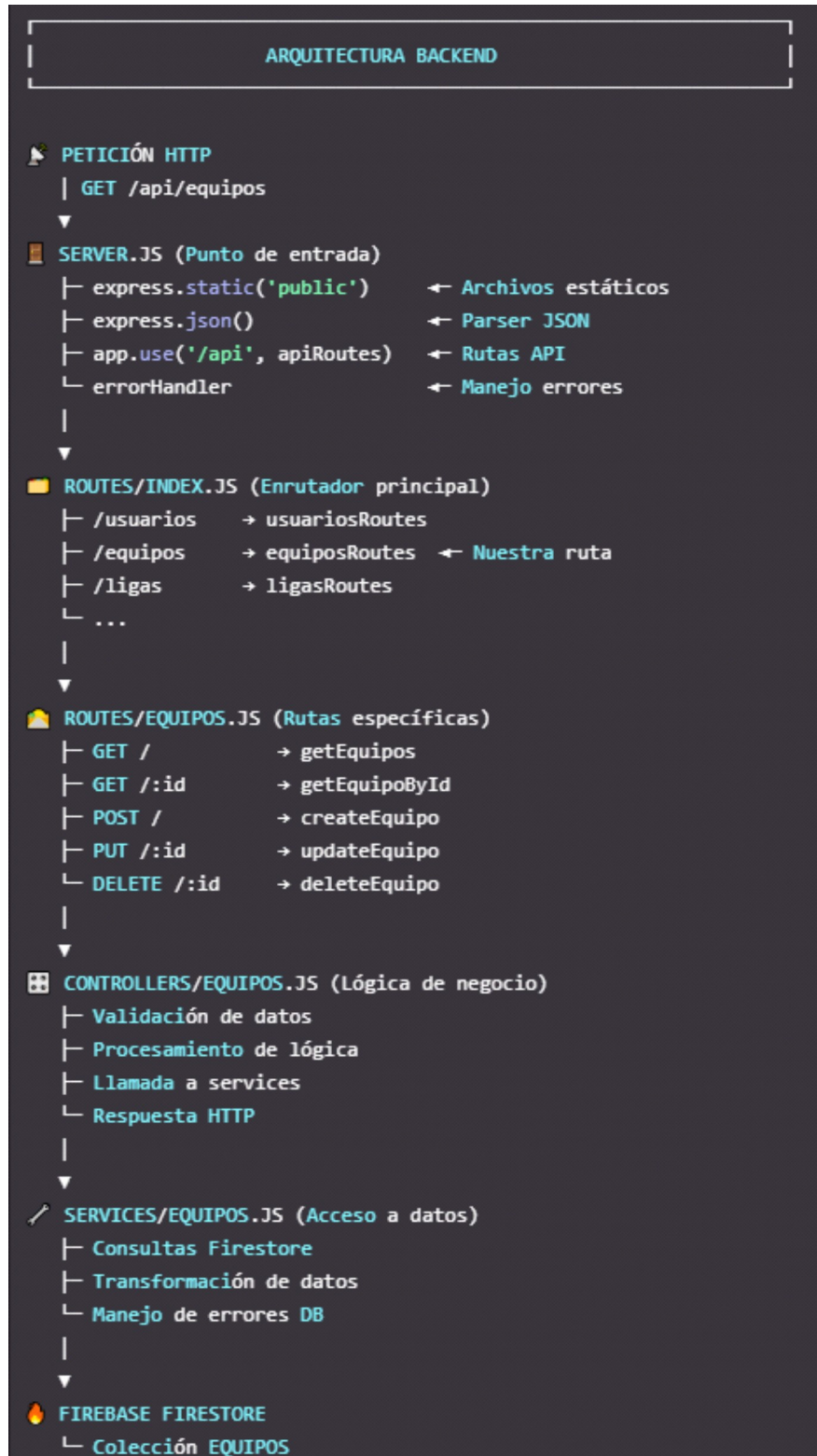
```
<!-- Loading State -->
<div class="text-center py-12">
  <i class="fas fa-spinner fa-spin text-4xl text-primary mb-4"></i>
  <p class="text-gray-500">Cargando datos...</p>
</div>

<!-- Empty State -->
<div class="text-center py-12">
  <i class="fas fa-[icono] text-6xl text-gray-300 mb-4"></i>
  <h3 class="text-xl font-semibold text-gray-600 mb-2">No hay datos</h3>
  <p class="text-gray-500">Mensaje explicativo</p>
</div>

<!-- Error State -->
<div class="text-center py-12">
  <i class="fas fa-exclamation-triangle text-6xl text-red-300 mb-4"></i>
  <h3 class="text-xl font-semibold text-red-600 mb-2">Error</h3>
  <p class="text-gray-500">Descripción del error</p>
</div>
```


3.4.2 Diseño “Backend”.

El Backend recurre a un servidor node.js para procesar las peticiones del cliente, transmitir las al servidor de la base de datos en la nube, cuando se recibe la respuesta del servidor de la base de datos, esta vuelve a ser procesada por node.js antes de ser enviada al usuario que la visualizará en su navegador. Nuevamente pensamos que una imagen vale más que mil palabras.



AUTENTIFICACIÓN JWT

Para comunicarse con el servidor y autenticar a los usuarios incorporamos la tecnología JWT, gracias a su estándar abierto se puede transmitir información segura entre dos partes como un objeto JSON firmado digitalmente. Gracias a un token JSON Web Token se puede verificar la autenticidad y la propiedad del paquete enviado al servidor. El token se genera cuando el usuario se logea en la web y el cliente solo tiene que incluirlo en todas sus peticiones para que el servidor las pueda reconocer. En nuestro caso se guarda en el Store del navegador del cliente. Además uno de los avances que hicimos fue cifrar las contraseñas en la base de datos de USUARIOS, al principio las guardábamos como texto plano pero al incorporar este código a la función de guardar los datos de la API conseguimos un gran resultado:

```
const bcrypt = require('bcrypt');

async function crearUsuario({ mail, password, userData = {} }) {

  const hash = await bcrypt.hash(password, 10);

  const toSave = {
    mail,
    contra: hash, // ← Se guarda el hash, NO la contraseña original
    ...userData
  };

  const docRef = await db.collection('USUARIOS').add(toSave);
}

bcrypt.hash() es la función de encriptación
```

¿Qué hace?

password es la contraseña en texto plano

10 son los salt rounds ($2^{10} = 1,024$ iteraciones)

hash es el resultado encriptado que se almacena en la base de datos

API

Ha sido el gran aporte y reto para este proyecto, surgió de una recomendación por parte del tutor y nos pusimos manos a la obra, su esquema parece sencillo pero tiene una enorme complejidad.

3.4.2.1 Diseño BBDD.

Diseño de la estructura de la base de datos NOSQL esta albergada en FirebaseStorage Database, no podemos mostrar nada más que el panel de acceso.

Básicamente en una base de datos que alberga colecciones de documentos:

La base de datos Firestore está organizada en las siguientes colecciones principales:

- **Colección: USUARIOS**

- **Propósito:** Almacena la información de los usuarios de la aplicación. Cada documento dentro de esta colección representa un usuario individual.
- **Estructura de un documento de la colección USUARIO :**
 - Cada documento tiene **un userId único, que no se puede repetir** (ejemplo: 2RSxpSdjGHgSCcDr5WIs).
 - Contiene los campos de:
 - nombre (string)
 - apellido1 (string)
 - apellido2 (string)
 - email (string)(que pasará a ser el nombre de usuario de la aplicación web)
 - contra (string) (donde se almacena la contraseña cifrada)
 - móvil (number)
 - numeroDocumento (string) (DNI, NIE, se valida en el propio formulario de registro de usuario)
 - roles (administrador, delegado, arbitro y entrenador).
- **Reglas de Acceso:**
 - allow create: if request.auth != null; : Solo los usuarios autenticados pueden crear nuevos documentos de usuario.
 - allow update, delete: if request.auth != null; : Solo los usuarios autenticados pueden actualizar o eliminar documentos de usuario (es común refinar esto para que solo el propietario del documento pueda modificarlo, por ejemplo if request.auth.uid == userId).
- **Colección: EQUIPOS**
 - **Propósito:** Contiene información sobre los equipos inscritos en la asociación deportiva y datos de alta en la web, estén o no estén inscritos a una liga.
 - **Estructura de un documento de la colección EQUIPOS :**
 - Cada documento tiene un equipoId único (por ejemplo 2pvDSwCte3E4CLe5RCn2).
 - Contiene los campos: EQUIPO (nombre del equipo), CATEGORÍA, TIPO (masculino, femenino o mixto).
 - **Reglas de Acceso:**

- allow read: true; : **Cualquier persona puede leer los documentos de esta colección.**
- allow create: request.auth != null; : Solo los usuarios autenticados pueden crear nuevos equipos.
- allow update, delete: request.auth != null; : Solo los usuarios autenticados pueden actualizar o eliminar equipos (similar a USUARIOS , esto podría requerir una lógica más granular).
- **Colección: CATEGORIAS**
 - **Propósito:** Almacena diferentes categorías que se utilizan para dividir a los equipos según la edad fundamentalmente.
 - **Estructura de un documento CATEGORÍA :**
 - Cada documento tiene un categoriaId único.
 - Contiene los campos:
 - NOMBRE (string)
 - ano1 (number)
 - ano2 (number)
 - EdadMin (number) (edad mínima)
 - EdadMax (number)(edad máxima para jugar en esa categoría).
 - **Reglas de Acceso:**
 - allow read: true; : Cualquier persona puede leer los documentos de esta colección.
 - allow create: request.auth != null; : Solo los usuarios autenticados pueden crear nuevas categorías.
 - allow update, delete: request.auth != null; : Solo los usuarios autenticados pueden actualizar o eliminar categorías (a menudo, las categorías son gestionadas por administradores, lo que requeriría una condición adicional en las reglas).
- **Colección LIGAS:**
 - **Propósito:** Almacena las diferentes competiciones que gestiona la web.
 - **Estructura de un documento LIGAS:**
 - Cada documento tiene un LigaId único.
 - Contiene los campos:
 - CALENDARIO (true/false)

- CATEGORÍA (es un string que se coge de la colección categorías).
- CATEGORIAID (string) (permite mayor seguridad a la hora de trabajar editando datos).
- EQUIPOS (COLECCIÓN) es una colección de documentos dentro de la propia colección, según los expertos no se debe abusar para evitar problemas en la ejecución. REALMENTE CONTIENE UN ARRAY de strings CON TODOS LOS EQUIPOS INSCRITOS EN LA LIGA.
- FECHA_FIN (timestamp)
- FECHA_INICIO (timestamp)
- NOMBRE (string), contiene el nombre de la liga.
- NUM_EQUIPOS (number) número de equipos en la liga.
- TEMPORADA (string)
- TIPO (string) este campo y el de categoría se heredan al crear el registro según lo seleccionado por el usuario.

- **Reglas de Acceso:**

- allow read: true; : Cualquier persona puede leer los documentos de esta colección.
- allow create: request.auth != null; : Solo los usuarios autenticados pueden crear nuevas ligas.
- allow update, delete: request.auth != null; : Solo los usuarios autenticados pueden actualizar o eliminar ligas si una liga tiene calendario de partidos asociado la API no deja borrarla.

- **Colección PARTIDOS:**

- **Propósito:** Almacena los partidos de las diferentes ligas, es probablemente la columna vertebral del proyecto junto con la gestión de usuarios de la web.
- **Estructura de un documento de la colección PARTIDOS:**
 - Cada documento tiene un partidoId único.
 - Contiene los campos:
 - AMARILLASLOCAL (number)
 - AMARILLASVISITANTE (number)
 - ARBITRO (string) contiene el valor id de un usuario con el rol arbitro que es asignado automáticamente de forma aleatoria al generarse el calendario de partidos de la liga.

- CAMPO (string) se extrae de la colección campos.
- CONERLOCAL (number)
- CORNERVISITANTE (number)
- FALTASLOCAL (number)
- FALTASVISITANTE (number)
- FECHA (timestamp) contiene el día y la hora a la que se celebrará el partido.
- GOLESLOCAL (number)
- GOLESVISITANTE (number)
- JORNADA (number)
- LIGA (string) se extrae de la colección ligas.
- ROJASLOCAL (number)
- ROJASVISITANTE (number)
- TIEMPOJUEGO (number)
- VISITANTE (string) se extrae de la colección equipos de la liga y se asignan de forma aleatoria ROUND ROBIN (en sentido contrario a las agujas del reloj para poder emparejar a todos los equipos y asignar fechas y horas de partidos).
- Estado: (string) puede ser finalizado, suspendido y programado.
- FechaCreacion (timestamp) es un valor de seguridad para futuras implementaciones, se graba cuando el calendario de partidos es creado.
- FechaModificacion (timestamp) con el valor de la última modificación realizada en el partido, suele ser la fecha y hora cuando el arbitro introduce el acta con el resultado.

- **Reglas de Acceso:**

- allow read: true; : Cualquier persona puede leer los documentos de esta colección.
- allow create: request.auth != null; : Solo los usuarios autenticados pueden crear nuevas ligas.
- allow update, delete: request.auth != null; : Solo los usuarios autenticados pueden actualizar o eliminar ligas si una liga tiene calendario de partidos asociado la API no deja borrarla.

- **Colección CAMPOS**

- **Propósito:** Almacenan los diferentes nombres de los campos donde se juegan los partidos.
- **Estructura de un documento de la colección PARTIDOS:**
 - Cada documento tiene un campoId único.
 - Contiene los campos:
 - CAMPO (string) (valga la redundancia contiene el nombre el campo).
 - AFORO (number)
 - NUM (number) numero de campo dentro de todos los de la asociación.
- **Reglas de Acceso:**
 - allow read: true; : **Cualquier persona puede leer los documentos de esta colección.**
 - allow create: request.auth != null; : Solo los usuarios autenticados pueden crear nuevas campos.
 - allow update, delete: request.auth != null; : Solo los usuarios autenticados pueden actualizar o eliminar CAMPOS. Se puede eliminar un campo, los partidos donde se han celebrado encuentros no se verán afectados, pero el campo dejará de estar disponible para ser seleccionado.

- **Colección TIPO**

- **Propósito:** Almacenan los diferentes nombres de los campos donde se juegan los partidos.
- **Estructura de un documento de la colección TIPO:**
 - Cada documento tiene un partidosId único.
 - Contiene el campo:
 - TIPO (string) puede albergar los valores MASCULINO, FEMENINO Y MIXTO).
- **Reglas de Acceso:**
 - allow read: true; : **Cualquier persona puede leer los documentos de esta colección.**
 - ESTA COLECCIÓN NO SE PUEDE MODIFICAR.

- **Colección mensajes**

- **Propósito:** Almacena registros que se utilizan para ponerse en contacto con los

administradores del sitio.

- **Estructura de un documento de la colección mensajes:**

- Cada documento tiene un mensajeId único.
- Contiene los campos:
 - asunto (string) (como cualquier mail que se tercie)
 - mail (string)
 - estado (string) puede tener los valores pendiente o leído.
 - FechaCreación (timestamp)
 - fechaLectura (timestamp)
 - leído (boolean)
 - mensaje (string)
 - nombre (string)

- **Reglas de Acceso:**

- allow create: request.auth != null; : cualquier usuario puede hacer un registro a través del formulario contacto.html (siempre hay que aceptar las cookies para poder guardar el mensaje en la colección).
- allow update, delete: request.auth != null; : Solo los usuarios autenticados pueden actualizar o eliminar CAMPOS. Se puede eliminar un campo, los partidos donde se han celebrado encuentros no se verán afectados, pero el campo dejará de estar disponible para ser seleccionado.

- **Colección JUGADORES:**

- **Propósito:** contiene los datos

- **Estructura de un documento de la colección PARTIDOS:**

- Cada documento tiene un partidoId único.
- Contiene los campos:
 - ALIAS (string)

- APELLIDO1 (string)
- APELLIDO2 (string)
- CATEGORIA (string)
- DOCUMENTO (string)
- DORSAL (number)
- EQUIPO (string)
- EQUIPO_ID (string)
- ESTADO (string)

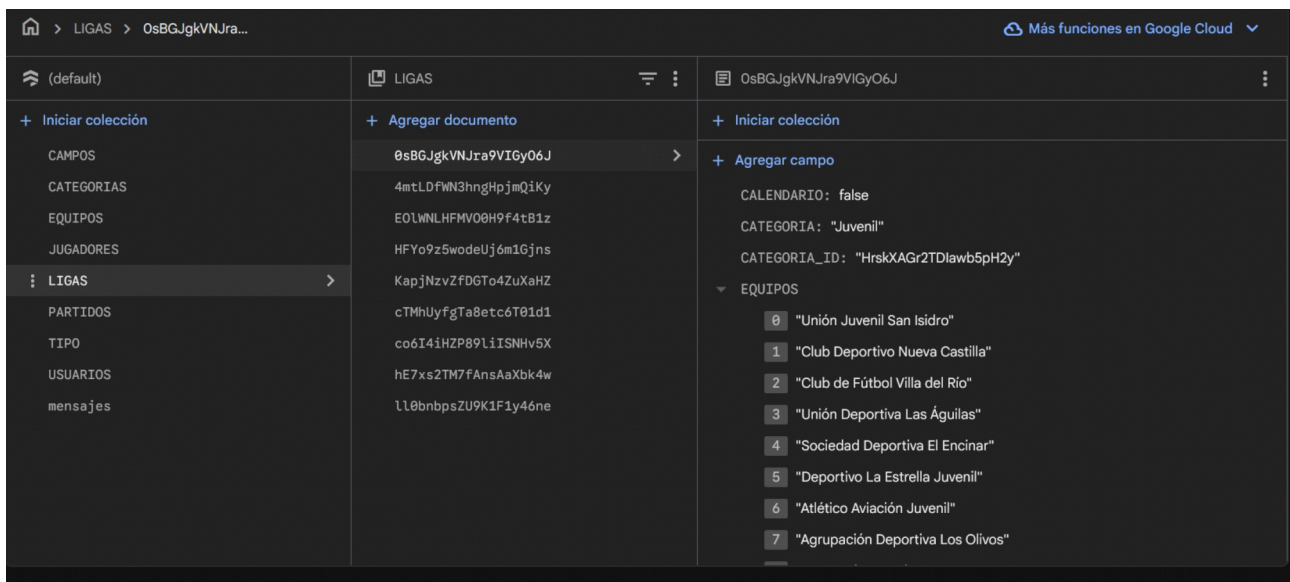
- FECHA_NACIMIENTO (timestamp)
- MAIL (string)
- MOVIL (number)

- **Reglas de Acceso:**

- allow create: request.auth != null; : Solo los usuarios autenticados pueden crear nuevos jugadores.
- allow update, delete: request.auth != null; : Solo los usuarios autenticados pueden actualizar o eliminar jugadores.

Como la base de datos no es relacional los datos se pueden grabar múltiples veces en distintos sitios valiéndonos de listas cerradas en un formulario donde el usuario no puede interactuar libremente solo con las opciones que se le dan.

Vista panel principal de FirebaseStore de la base de datos como si fuera un phpMyadmin pero el propio servidor en la nube, el problema es que hay que estar logeado en Google para poder acceder.



3.5 Pruebas.

En este proyecto se han realizado muchas pruebas de uso, funcionalidad y ejecución. Todos los datos de la base de datos han sido introducidos usando los formularios de la propia web, cuando se ha creado una nueva funcionalidad se ha implementado y probado con los propios elementos de la web. El desarrollo se ha hecho en local en los equipos personales del equipo pero la en todo momento los datos de todas las colecciones han sido introducidos de forma manual. Se podría haber utilizado otros elementos de IA para realizar esta gestión pero al haberlo hecho manualmente se han podido ver y depurar los errores en su manejo y utilización.

3.6 Despliegue de la aplicación.

El despliegue de la aplicación es sencillo pero largo, al activar el HOSTING se instalan las

dependencias para poder comunicarte con el servidor de la base de datos y se instala una API básica que Firebase ofrece de forma gratuita, para controlar el acceso se facilita una clave privada en forma de archivo JSON donde se puede ver el siguiente contenido:

```
{
  "type": "service_account",
  "project_id": "de-la-paz-42126976-5cd58",
  "private_key_id": "a6a310aae545e7ecea8197cc7f2149ab8ea05384",
  "private_key": "-----BEGIN PRIVATE KEY-----\nMIIEvAIBADANBgkqhkiG9w0BAQE\n"client_email": "firebase-adminsdk-fbsvc@de-la-paz-42126976-5cd58.iam.g\n"client_id": "117897240158628183320",
  "auth_uri": "https://accounts.google.com/o/oauth2/auth",
  "token_uri": "https://oauth2.googleapis.com/token",
  "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/cer\n"client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x5\n"universe_domain": "googleapis.com"
}
```

Por seguridad no se muestra toda la `private_key`. Gracias a esta clave la web se puede comunicar con la base de datos.

El servidor NODE.JS requiere de la actualización del plan para el despliegue, ofrecen un crédito de 300\$ de bienvenida y solo se cobra por uso (cada vez que un usuario accede al sitio web) por lo tanto es muy práctico, lamentablemente no hemos podido despegarlo totalmente dado que habría que hacer modificaciones en la API de la web para utilizar el nuevo servidor NODE.JS en la nube en lugar de el servidor en local que hemos utilizado en el desarrollo.

En cuanto a **TailwindCSS** la instalación es rapidísima se puede hacer copiando las dependencias directamente en el sitio web o bien importándolas del repositorio.

JWT fue un paso importante, después de instalar las librerías se debe incluir la función para el inicio de sesión:

```
const jwt = require("jsonwebtoken");

function login(req, res) {
  const { email, password } = req.body;

  // 1. Validar usuario en tu base de datos
  const user = buscarUsuario(email, password);
  if (!user) return res.status(401).json({ error: "Credenciales inválidas" });

  // 2. Crear token
  const token = jwt.sign(
    { userId: user.id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: "2h" }
  );
}
```

```
// 3. Enviar token al cliente
```

```
res.json({ token });
```

```
}
```

El cliente guarda el token en su localStorage en sessionStorage y hay que hacer que el cliente envíe el token en cada petición:

```
fetch("/api/equipos", {
```

```
  headers: {
```

```
    "Authorization": "Bearer " + localStorage.getItem("token")
```

```
  }
```

```
});
```

Middleware en el servidor para verificar el token:

```
function authMiddleware(req, res, next) {
```

```
  const header = req.headers["authorization"];
```

```
  if (!header) return res.status(401).json({ error: "Token requerido" });
```

```
  const token = header.split(" ")[1];
```

```
  jwt.verify(token, process.env.JWT_SECRET, (err, decoded) => {
```

```
    if (err) return res.status(403).json({ error: "Token inválido" });
```

```
    req.user = decoded; // userId, role, etc.
```

```
    next();
```

```
  });
```

```
}
```

Se protegen las rutas:

```
app.get("/api/equipos", authMiddleware, (req, res) => {
```

```
  res.json({ mensaje: "Acceso permitido", user: req.user });
```

```
});
```

Y ya se puede usar.

En cuanto al despliegue del HOSTING Firebase según su creador Google hay que dar estos pasos SEGÚN SU ASISTENTE PERSONAL, O ESO DICEN:



- **Configurar Firebase:**

- Crea un proyecto nuevo en la consola de Firebase.
- Activa **Firestore** como tu base de datos.
- En la configuración de tu proyecto, ve a **Cuentas de servicio** y genera una nueva

clave privada. Esto descargará un archivo JSON que deberás añadir a tu proyecto.

- **Preparar tu Entorno Local:**

- Asegúrate de tener **Node.js** instalado (versión 20.x recomendada).
- Descarga o clona el código de tu proyecto.
- Coloca el archivo JSON que descargaste de Firebase en la raíz de tu proyecto y renombrado a `firebase-adminsdk.json`.
- Abre una terminal en la raíz de tu proyecto y ejecuta `npm install` para instalar todas las dependencias.

- **Adaptar el Servidor para Firebase:**

- Necesitamos preparar tu servidor de Express para que funcione como una **Cloud Function** en lugar de un servidor tradicional.
- Instala las dependencias de Firebase con `npm install firebase-functions firebase-admin`.
- Modifica tu archivo `server.js` para exportar la aplicación de Express usando `functions.https.onRequest(app)`.

- **Configurar el Despliegue:**

- Ajusta el archivo `firebase.json` para indicarle a Firebase que las peticiones a tu API deben ser manejadas por tu Cloud Function.

- **Desplegar el Proyecto:**

- Finalmente, ejecuta el comando `firebase deploy` desde tu terminal para subir todo a Firebase.

SOLO NOS FALTA EL SERVIDOR NODE.JS, y adaptar la API.

4. Resultados y discusión.

El resultado de este proyecto ha sido una aplicación web HÍBRIDA a la que llamaremos nuestro pequeño FRANKSTEIN. Por un lado el repositorio de GITHUB esta conectado con el servicio de HOSTING, la base de datos se ha creado y es operativa y accesible a través de la red. Por otro lado la pieza fundamental del proyecto es la API propia y el servidor NODE.JS que son el alma de todo el proceso sin ellos no ha web ni acceso a los datos, podríamos haber implementado el acceso a través de la API de Firebase que es no es muy buena, pero el servidor NODE.JS era fundamental sin él no hay operatividad. Nos habría encantado poner a funcionar el servidor NODE.JS ha funcionar, configurarlo y usarlo pero solo con el consumo que hemos hecho para desarrollar el proyecto habríamos agotado el crédito pasando a un plan de pago que es lo que Google quería desde el primer momento.

Por otro lado recomendamos Firebase como HOSTING para páginas web estáticas, es gratuito y se puede usar sin compromiso, permite tener muchos proyectos y es 100% compatible con GITHUB. Las tecnologías que hemos cogido son nuevas para nosotros y sobre todo trabajar con NODE.JS ha sido toda una novedad, el desarrollo de la API ha sido un reto muy grande pero al final con la ayuda de la IA lo hemos conseguido.

5. Conclusiones.

Recomendamos el uso de la IA acorta los tiempos sorprendentemente, pero tiene sus limitaciones, debes decirle lo que quieres de una forma profesional o técnica. Gracias a la metodología SCRUM fuimos incorporando funcionalidad al proyecto poco a poco hasta ir consiguiendo nuevas funciones y más robustez.

Para nosotr@s ha sido un placer formar equipo y abordar los retos del proyecto, reconocemos que hemos sido muy osados.

Darle las gracias a todos los profesores del tribunal y especialmente a nuestro tutor Jorge Iván Rodríguez que nos ha sido de una gran ayuda, sin el no lo habríamos conseguido.

6. Bibliografía y referencias.

Sitios oficiales

Firebase — Plataforma oficial de desarrollo web y móvil

<https://firebase.google.com>

Firebase Documentation — Documentación oficial de Firebase

<https://firebase.google.com/docs>

Node.js — Sitio web oficial del entorno de ejecución

<https://nodejs.org>

Node.js Documentation — Documentación oficial de la API

<https://nodejs.org/en/docs>

TailwindCSS — Sitio oficial del framework

<https://tailwindcss.com>

TailwindCSS Documentation — Documentación oficial completa

<https://tailwindcss.com/docs>

TailwindCSS GitHub — Repositorio oficial del proyecto

<https://github.com/tailwindlabs/tailwindcss> (github.com in Bing)

TailwindCSS: Instalación y primeros pasos (Manual oficial)

<https://tailwindcss.com/docs/installation> (tailwindcss.com in Bing)

freeCodeCamp — Tutorial práctico de TailwindCSS

<https://www.freecodecamp.org/news/how-to-use-tailwind-css/>

(freecodecamp.org in Bing)

MDN Web Docs — Introducción a frameworks CSS (incluye Tailwind)

https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Frameworks (developer.mozilla.org in Bing)

JWT.io — Sitio oficial del estándar JWT

<https://jwt.io>

JWT.io Introduction — Explicación oficial del funcionamiento de JWT

<https://jwt.io/introduction> (jwt.io in Bing)

RFC 7519 — Especificación oficial de JSON Web Token

<https://www.rfc-editor.org/rfc/rfc7519>

Auth0 — Guía completa de JWT (muy recomendada)

<https://auth0.com/learn/json-web-tokens> (auth0.com in Bing)

Node.js + JWT (Tutorial oficial de Auth0 para APIs)

<https://auth0.com/blog/node-js-and-express-tutorial-building-and-securing-restful-apis/> (auth0.com in Bing)

freeCodeCamp — Cómo implementar JWT en Node.js

<https://www.freecodecamp.org/news/how-to-sign-and-verify-json-web-tokens/> (freecodecamp.org in Bing)

DigitalOcean — Tutorial JWT con Node.js y Express

<https://www.digitalocean.com/community/tutorials/nodejs-jwt-expressjs> (digitalocean.com in Bing)

Tutoriales y manuales para crear APIs con Node.js

freeCodeCamp — Crear una API con Node.js y Express (curso completo)

<https://www.freecodecamp.org/espanol/news/crear-una-api-rest-con-node-js-y-express/>
(freecodecamp.org in Bing)

Urian Viera — Cómo crear una API con Node.js y Express (guía práctica)

<https://urianviera.com/como-crear-una-api-con-nodejs-y-express/>
(urianviera.com in Bing)

KeepCoding — Cómo crear una API REST con Node.js y Express

<https://keepcoding.io/blog/como-crear-una-api-rest-con-node-js-y-express/> (keepcoding.io in Bing)

NanoTutoriales — API RESTful con Node.js y Express (tutorial completo)

<https://nanotutoriales.com/api-rest-nodejs-express/> (nanotutoriales.com in Bing)

Tutoriales sobre APIs conectadas a Firebase

Guía completa: Node.js + Express + Firebase (API moderna)

<https://www.freecodecamp.org/news/nodejs-express-firebase-api-guide/> (freecodecamp.org in Bing)

YouTube — Conectar una API Node.js con Firebase (CRUD completo)

<https://www.youtube.com/watch?v=READ> (youtube.com in Bing)

Firebase Admin SDK para Node.js — Referencia oficial

<https://firebase.google.com/docs/admin/setup> (firebase.google.com in Bing)

Recursos adicionales útiles

GitHub — Quickstart oficial de Firebase para Node.js

<https://github.com/firebase/quickstart-nodejs> (github.com in Bing)

Guía oficial: Añadir Firebase a un proyecto web

<https://firebase.google.com/docs/web/setup> (firebase.google.com in Bing)

ANEXOS

ANEXO A

Guía del Diseño Visual de la pagina Web

<https://github.com/FranTieso/De-la-Paz/blob/main/Guia%20Dise%C3%B1o%20de%20la%20Web.pdf>

ANEXO B

Guía del Diseño de clases

https://github.com/FranTieso/De-la-Paz/blob/main/diagrama_de_clases.png

ANEXO C

Descripción general

API REST desarrollada con Node.js + Express, conectada a Firebase (Firestore) mediante Firebase Admin. Esta API da soporte a la aplicación web de gestión deportiva del proyecto DAW.

Gestiona:

Usuarios y autenticación

Ligas, equipos y jugadores

Partidos, resultados y clasificaciones

Calendario, campos y mensajes

URL base: <http://localhost:3001/api>

Autenticación y seguridad

Autenticación

JWT (JSON Web Token)

Header obligatorio: Authorization: Bearer

Middleware:

auth.js

Roles

admin

delegado

entrenador

El acceso a los endpoints está controlado mediante middleware de permisos.

Endpoints

Usuarios / Auth
POST /usuarios/login
GET /usuarios
GET /usuarios/:id
POST /usuarios
PUT /usuarios/:id
DELETE /usuarios/:id

Ligas

GET /ligas
GET /ligas/:id
POST /ligas
PUT /ligas/:id
DELETE /ligas/:id

Equipos

GET /equipos
GET /equipos/:id
POST /equipos
PUT /equipos/:id
DELETE /equipos/:id

Jugadores

GET /jugadores
GET /jugadores/:id
POST /jugadores
PUT /jugadores/:id
DELETE /jugadores/:id

Partidos

GET /partidos
GET /partidos/:id
POST /partidos
PUT /partidos/:id
DELETE /partidos/:id

Resultados

GET /resultados
POST /resultados

Clasificaciones

GET /clasificaciones

Calendario

GET /calendario

Campos

GET /campos

POST /campos

Mensajes

GET /mensajes

POST /mensajes

Middlewares

auth.js → Verificación de JWT

permissions.js → Control de roles

validator.js → Saneado y validación de datos

errorHandler.js → Gestión centralizada de errores

Códigos de estado HTTP

200 OK

201 Created

400 Bad Request

401 Unauthorized

403 Forbidden

404 Not Found

500 Internal Server Error

Arquitectura Backend

Estructura de carpetas

api/ |— config/ | — firebase.js |— controllers/ | |— usuariosController.js | |— ligasController.js | |— equiposController.js | |— jugadoresController.js | |— partidosController.js | |— resultadosController.js | |— clasificacionesController.js | |— calendarioController.js | |— camposController.js | — mensajesController.js |— middlewares/ | |— auth.js | |— permissions.js | |— validator.js | — errorHandler.js — routes/ — index.js

Flujo de una petición

Cliente

→ Router

→ Middleware de autenticación

→ Middleware de permisos

→ Controller

- Firestore
- Respuesta HTTP

Base de datos

Base de datos:

Firestore (NoSQL)

Colecciones:

usuarios
ligas
equipos
jugadores
partidos
resultados
clasificaciones
mensajes
campos

Ventajas de la arquitectura

Arquitectura modular
Escalable y mantenible
Separación clara de responsabilidades
Preparada para ampliaciones futuras